

AI-gestuurd dependency dashboard voor automatische analyse en prioritering van software-updates.

Jelle Vandriessche.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. T. Aelbrecht

Co-promotor: Dhr. B. Pattyn

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Voor u ligt mijn bachelorproef, waarin ik onderzoek heb gedaan naar de mogelijkheden van een AI-gestuurd dashboard voor dependencybeheer. Het onderwerp sprak mij bijzonder aan omdat het een combinatie vormt van softwareontwikkeling, AI-interactie en praktische probleemoplossing binnen een echte bedrijfscontext. Tijdens dit traject heb ik niet alleen mijn technische kennis verder verdiept, maar ook ervaren hoe belangrijk het is om een helder onderzoeksdoel te koppelen aan een bruikbare implementatie.

Deze bachelorproef was voor mij een leerzaam en uitdagend proces. Het uitwerken van de requirements, het ontwerpen van de architectuur, het bouwen van het proof-of-concept en het vergelijken van verschillende AI-aanpakken vroegen telkens om nieuwe inzichten en nauwkeurige keuzes. Vooral de vertaalslag van een conceptueel idee naar een werkende toepassing heb ik als bijzonder waardevol ervaren.

Graag wil ik enkele mensen bedanken die mij tijdens dit traject hebben ondersteund. In de eerste plaats dank ik mijn promotor, Thomas Aelbrecht, voor de begeleiding bij het schrijven en structureren van de paper, en voor de waardevolle feedback die mij geholpen heeft om mijn werk scherper en consistentier uit te werken. Daarnaast wil ik mijn co-promotor, Bert Pattyn, bedanken voor de inhoudelijke en praktische ondersteuning binnen Springbok zowel tijdens het papertraject als bij het uitwerken van de bachelorproef zelf. Ook gaat mijn dank uit naar Michiel Bogaert, die mij binnen Springbok heeft geholpen met het praktische gedeelte van het project en mij gerichte feedback gaf op mijn code en implementatie. Hun begeleiding en input hebben in belangrijke mate bijgedragen aan het resultaat van deze bachelorproef.

Tot slot wil ik ook mijn omgeving bedenken voor de steun en het geduld tijdens dit traject. Hun aanmoediging heeft mij geholpen om dit project met volharding af te ronden

Jelle Vandriessche

Samenvatting

Deze bachelorproef onderzoekt in welke mate een AI-gedreven systeem kan worden ingezet om dependency-updates automatisch te analyseren en te prioriteren binnen een dashboardomgeving. Het ontwikkelde proof of concept, DepGuard AI, bestaat uit een aparte webapplicatie die project- en dependencygegevens ophaalt uit Github en publieke registries zoals npm, en die deze gegevens combineert met AI-gegenereerde samenvattingen van release notes en changelogs. De applicatie is ontworpen als losstaand dashboard bovenop een backendlaag, zodat authenticatie, validatie foutafhandeling en opslag centraal beheerd kunnen worden.

Binnen het proof of concept werden drie AI-aanpakken uitgewerkt en met elkaar vergeleken: een Mastra-gebaseerde agent, een n8n-workflow en een eigen OpenAI-agent met toolloop. De evaluatie gebeurde aan de hand van vijf representatieve testcase-updates en een LLM-as-a-judge benchmark die de output beoordeelde op specificiteit, compleetheid en actionability. De resultaten tonen aan dat de drie AI varianten bruikbare samenvattingen kunnen genereren, maar dat er duidelijke verschillen bestaan in kwaliteit, consistentie en snelheid. Mastra behaalde de hoogste scores op inhoudelijke kwaliteit en stabiliteit, terwijl OpenAI sneller was en n8n het laagst scoorde op specificiteit en volledigheid.

Op basis van de implementatie en evaluatie kan worden besloten dat een AI-gestuurde benadering een relevante meerwaarde biedt voor dependencybeheer, omdat ze de interpretatielast van changelogs en release notes verlaagt. Tegelijk toont de studie aan dat de kwaliteit van de uitkomst sterk afhangt van de gekozen orchestratie en van de beschikbaarheid van actuele broninformatie. Voor de context van Springbok vormt Mastra daardoor de meest geschikte basis voor verdere uitwerking van het dependency-dashboard.

Inhoudsopgave

Lijst van figuren	viii
Lijst van tabellen	ix
Lijst van codefragmenten	x
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	3
1.4 Opzet van deze bachelorproef	4
2 Stand van zaken	6
2.1 Dependency-updates als onderhoudslast	6
2.1.1 Kwetsbare en verouderde dependencies in de praktijk	6
2.1.2 Technische schuld als gevolg van uitgesteld onderhoud	7
2.1.3 De kloof tussen risico en actie	7
2.2 Bestaande oplossingen en hun grenzen	8
2.2.1 Dependencybots: wat ze wel en niet doen	8
2.2.2 De ontbrekende schakel: context en beslissingsondersteuning	8
2.2.3 Bijzondere uitdagingen bij private repositories en registries	9
2.3 AI-agents en workflow automatisering	9
2.3.1 Geschiktheid van een taak voor een AI-agent	9
2.3.2 Agentic design: tools, orkestratie en feedbackloops	10
2.3.3 Code gedreven versus low-code orkestratie	10
2.3.4 Tavily als zoek- en extractielaag	10
2.4 Platformen en ontwerpkeuzes	11
2.4.1 Code-gedreven frameworks	12
2.4.2 Low-code workflowtools	12
2.4.3 API-gestuurde agentconfiguratie	12
2.4.4 Vergelijkend overzicht van platformcategorieën	13
2.5 Synthese: de onderzoeksniche	13
3 Methodologie	15
3.1 Fase 1 - Verdiepende literatuurstudie en probleemverkenning	15
3.2 Fase 2 - Requirements-analyse en architectuurontwerp	16
3.3 Fase 3 - Implementatie van het proof-of-concept	16

3.4	Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak .	17
3.5	Fase 5 - Evaluatie en rapportering	18
4	Requirementsanalyse	20
4.1	Stakeholders	20
4.2	Functionele vereisten	21
4.2.1	Authenticatie en gebruikersbeheer	21
4.2.2	Projectbeheer	21
4.2.3	Dependency scanning en versiebeheer	22
4.2.4	AI-analyse en samenvatting	22
4.2.5	Dashboard en visualisatie	23
4.2.6	Notificaties en communicatie	23
4.2.7	Regels en beslissingsondersteuning	24
4.3	Niet-functionele vereisten	24
4.3.1	Beveiliging	24
4.3.2	Prestaties	24
4.3.3	Schaalbaarheid	25
4.3.4	Betrouwbaarheid en foutafhandeling	25
4.3.5	Onderhoudbaarheid	25
4.3.6	Bruikbaarheid	25
4.3.7	Traceerbaarheid en audit	26
4.3.8	Portabiliteit en installatie	26
4.4	Samenvattende MoSCoW-prioritering	26
5	Ontwerp en implementatie van het AI-gestuurde dashboard	27
5.1	Architectuur van het systeem	27
5.2	Implementatie van de Mastra-agent	30
5.3	Implementatie van de n8n-workflow	34
5.4	Implementatie van de OpenAI-agent-builder	37
5.5	Integratie met de webapplicatie	40
6	Resultaten en evaluatie	43
6.1	Evaluatiekader	43
6.1.1	Scoremethode: LLM-as-a-Judge	43
6.1.2	Testcaseselectie en verwachte output	45
6.2	Globale resultaten	46
6.2.1	Statistische vergelijking van de agents	48
6.3	Analyse per testcase	48
6.3.1	React minor (18.2.0 → 18.3.0)	48
6.3.2	Next.js minor (14.1.0 → 14.2.0)	49
6.3.3	TypeScript major (4.9.5 → 5.0.4)	49
6.3.4	Zod patch (3.22.3 → 3.22.4)	49
6.3.5	Tailwind CSS minor (3.3.0 → 3.4.0)	50

6.4	Analyse per agent	50
6.5	Interpretatie van de resultaten	51
7	Conclusie	52
A	Onderzoeksvoorstel	54
A.1	Inleiding	54
A.1.1	Probleemstelling	54
A.1.2	Onderzoeksvraag	55
A.1.3	Onderzoeksdoelstelling	56
A.2	Literatuurstudie	57
A.2.1	Dependency management en kwetsbare dependencies	57
A.2.2	Technische schuld en onderhoudslast	58
A.2.3	Dependencybots en hun beperkingen	58
A.2.4	AI, agents en workflow automatisering	58
A.2.5	Onderzoeksniche: AI-ondersteund dependencybeheer	59
A.3	Methodologie	60
A.3.1	Fase 1 - Verdiepende literatuurstudie en probleemverkenning.	60
A.3.2	Fase 2 - Requirements-analyse en architectuurontwerp	60
A.3.3	Fase 3 - Implementatie van het proof-of-concept	61
A.3.4	Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak.	61
A.3.5	Fase 5 - Evaluatie en rapportering	62
A.4	Verwacht resultaat en conclusie	63
	Bibliografie	65

Lijst van figuren

3.1	Fasering en tijdsplanning van het onderzoek	19
5.1	Architectuuroverzicht DepGuard AI	28
5.2	Conceptueel datamodel DepGuard AI	30
5.3	AI-workflow in n8n	35
5.4	Dashboardoverzicht van DepGuard AI met projectbeheer en statusin- dicatoren per repository.	40
5.5	Detailpagina van een project met de dependencylijst, versiestatus en beschikbare AI-analyses.	41
5.6	AI-samenvatting van een dependency-update, weergegeven samen met versie-informatie, updateclassificatie en changelog-verwijzing. . .	42
6.1	Overallscores per testcase	47
6.2	Scores per beoordelingsdimensie	47
A.1	Fasering en tijdsplanning van het onderzoek	64

Lijst van tabellen

2.1	Vergelijkend overzicht van platformcategorieën voor AI-agenten (gebaseerd op Anthropic, 2025; Langflow, 2025; Nair, 2025; OpenAI, 2025; Turing, 2026)	13
6.1	Overzicht van de gebruikte testcases met verwacht veiligheidsoordeel	46
6.2	Samenvatting van de benchmarkresultaten per AI-variant (gemiddelden $\pm \sigma$ over 5 testcases $\times$ 3 runs)	46

Lijst van codefragmenten

5.1	Configuratie van de dependencyAgent met de nodige tools voor de gekozen research strategie	32
5.2	Codefragment: fetchNpmInfoTool met npm-metadata en fallbacklogica	33
5.3	Codefragment: webChangelogSearchTool voor webzoekopdrachten naar changelogs	34
5.4	System prompt van de n8n Node 6 (Gemini AI-agent) met strikt outputformaat	36
5.5	Codefragment: Next.js-route die de n8n-workflow aanroept	37
5.6	OpenAI dependencyAgent met system prompt en tool-loop	38
5.7	fetchNpmInfoTool: npm-metadata en GitHub-releases met CHANGELOG-fallback	39
5.8	webChangelogSearchTool: webzoekopdrachten naar changelogs . . .	39

1

Inleiding

Softwareprojecten binnen bedrijven en organisaties maken gebruik van talrijke externe bibliotheken, frameworks en tools, de zogenaamde *dependencies*. Deze dependencies worden voortdurend bijgewerkt om nieuwe functionaliteiten, verbeterde prestaties en vooral beveiligingspatches aan te bieden (Pashchenko et al., 2018). In de praktijk blijkt het voor ontwikkelteams echter moeilijk om alle updates systematisch op te volgen en grondig te evalueren, zeker wanneer een bedrijf meerdere projecten en repositories beheert.

1.1. Probleemstelling

Het manueel opvolgen van dependency-updates vormt een groot probleem in softwareontwikkeling. Ontwikkelaars moeten regelmatig changelogs doorzoeken, compatibiliteit controleren en inschatten welke updates veilig kunnen worden doorgevoerd, wat in de praktijk veel tijd vraagt en vaak wordt uitgesteld (Behutiye et al., 2024; Pashchenko et al., 2018). Onderzoek toont aan dat veel softwareprojecten hun dependencies niet systematisch bijhouden, waardoor een aanzienlijk deel verouderd raakt Pashchenko et al. (2018). stellen bijvoorbeeld dat een belangrijk percentage kwetsbare dependencies relatief eenvoudig opgelost kan worden door een update, maar dat dit in de praktijk niet gebeurt wegens tijdsdruk en gebrek aan overzicht. Wanneer het updateproces niet systematisch gebeurt, stapelen onderhoudstaken zich op, waardoor het steeds moeilijker en tijdrovender wordt om de software up-to-date te houden. Dit fenomeen staat bekend als *technische schuld* (technical debt): de achterstand die ontstaat doordat noodzakelijke verbeteringen of updates te lang worden uitgesteld, wat leidt tot hogere onderhoudskosten en complexiteit op lange termijn Behutiye et al. (2024) en Ruiz et al. (2024). Empirisch onderzoek bevestigt dat technische schuld binnen bedrijven reële negatieve gevolgen heeft voor productiviteit en softwarekwaliteit Besker (2020). Om deze last te

verlichten, zetten veel organisaties al geautomatiseerde dependencybots in, zoals Dependabot¹ of Renovate.² Deze tools maken wel automatisch pull requests aan zodra er nieuwe versies beschikbaar zijn, Erlenhov et al. (2022) maar geven doorgaans weinig context over de inhoud en impact van de wijziging. Ontwikkelaars verliezen daardoor nog steeds tijd aan het interpreteren van changelogs, het inschatten van risico's en het beslissen of een update al dan niet moet worden doorgevoerd Erlenhov et al. (2022). Deze bachelorproef richt zich daarom op het ontwikkelen van een *aparte webapplicatie* (een dashboard) die integreert met GitHub-repositories van Springbok. Per project houdt deze applicatie de gebruikte dependencies bij, controleert via publieke registries (zoals npm³) of er nieuwe versies beschikbaar zijn en verzamelt de relevante changelogs en release notes. Een AI-agent, die in dit onderzoek wordt ontworpen op basis van bestaande AI-modellen maar zonder voorafgebouwd agentframework, analyseert deze informatie, genereert begrijpelijke samenvattingen en beoordeelt in welke mate een update cruciaal is (bijvoorbeeld om veiligheidsredenen) of eerder optioneel. De AI-agent geeft het kernteam via het dashboard inzicht in vragen zoals: welke dependencies zijn verouderd, wat is er precies veranderd in de nieuwe versie, zijn er bekende beveiligingsrisico's als niet wordt geüpdatet, en lijkt de update technisch laag- of hoogrisicovol.

1.2. Onderzoeksvraag

Om het probleem van handmatig dependencybeheer en beperkte context bij updates op te lossen, wordt de volgende hoofdonderzoeksvraag geformuleerd:

Hoe kan een AI-gedreven agent worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren, zodat het kernteam dat verantwoordelijk is voor platform- en dependencybeheer efficiënter en met minder handmatig werk dependency-updates kan verwerken?

Hieruit vloeien de volgende deelvragen voort.

Deelvragen voor het beter begrijpen van het probleem

1. Wat zijn de belangrijkste uitdagingen en risico's bij het huidige, overwegend manuele dependency management voor het kernteam dat verantwoordelijk is voor platform- en dependencybeheer?

¹Dependabot is de ingebouwde GitHub-tool die kwetsbare of verouderde dependencies detecteert en automatisch update-pull-requests aanmaakt. <https://github.com/dependabot>

²Renovate is een open-source tool die automatisch update-pull-requests voor dependencies aanmaakt. <https://docs.renovatebot.com>

³npm is het standaard pakketbeheerplatform voor het JavaScript-ecosysteem. <https://www.npmjs.com>

2. Welke bestaande tools en oplossingen voor automatisch dependencybeheer worden vandaag gebruikt, en welke sterke punten en beperkingen hebben deze oplossingen in de context van Springbok?
3. Hoe leidt ophopende technische schuld op het vlak van dependency-updates ertoe dat er steeds meer tijd en inspanning nodig zijn voor het controleren en bijwerken van dependencies binnen Springbok?

Deelvragen richting de oplossing

4. Hoe kunnen AI-agents en workflow automatiseringsplatformen worden ingezet om informatie uit changelogs, release notes en dependency-pull-requests automatisch te interpreteren en in begrijpelijke vorm te communiceren naar het verantwoordelijke kernteam?
5. Welke functionele en niet-functionele vereisten moet een platform voor AI-gestuurd dependencybeheer vervullen, bijvoorbeeld op het vlak van integratie met versiebeheersystemen en registries, uitbreidbaarheid, beheer van regels en policies, auditlogging en performantie?
6. In welke mate voldoen geselecteerde AI-agent- of workflowplatformen aan deze vereisten, en hoe goed kunnen zij geïntegreerd worden in een proof-of-concept voor dependency management in de context van Springbok?
7. In welke mate presteren de geselecteerde AI-agents effectief bij het analyseren en prioriteren van dependency-updates, gemeten in termen van nauwkeurigheid, bruikbaarheid van de adviezen en vermindering van manueel werk voor het kernteam?

1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is om een proof-of-concept te ontwikkelen van een *aparte webapplicatie* (dashboard) die automatisch dependency-updates detecteert, analyseert en communiceert via een geïntegreerde AI-agent, en om na te gaan in welke mate geselecteerde AI-agent- of workflowplatformen deze oplossing kunnen ondersteunen en voldoen aan de vereisten van Springbok voor geautomatiseerd dependencybeheer. De webapplicatie fungeert daarbij als losstaand platform naast GitHub en gebruikt GitHub voornamelijk als bron voor project- en dependency-informatie en als kanaal om eventuele vervolgacties (zoals pull requests of issues) te initiëren.

Concreet zal de oplossing:

- per project de gebruikte dependencies bijhouden;
- automatisch nagaan of er nieuwe versies beschikbaar zijn via publieke registries (zoals npm);

- changelogs en release notes analyseren met behulp van een AI-agent die samenvat wat er gewijzigd is en welke risico's of voordelen een update inhoudt;
- de verantwoordelijke developers via een dashboard informeren over de aard en impact van de updates, inclusief een korte, begrijpelijke samenvatting en een indicatie of de update cruciaal, risicovol of veilig lijkt;
- regels ondersteunen waardoor de developers gemakkelijker kan beslissen of en wanneer een dependency geüpdatet wordt, zonder dat deze beslissingen automatisch in GitHub worden afgedwongen.

De concrete technologiekeuzes worden in de verdere uitwerking van de bachelorproef gemotiveerd op basis van de requirements en de context van Springbok. Het onderzoek resulteert in:

- een prototype (demo) dat de werking van het systeem aantoont binnen een of enkele projecten van Springbok;
- een evaluatie van de toegevoegde waarde van AI bij dependency management voor de specifieke developer;
- een aanbeveling welk type platform, en eventueel welke concrete technologie, het meest geschikt is voor verdere toepassing binnen Springbok.

1.4. Opzet van deze bachelorproef

De bachelorproef volgt een klassieke onderzoeksstructuur en is als volgt opgebouwd:

- **Hoofdstuk 1** Dit hoofdstuk beschrijft de context, de probleemstelling, de onderzoeksvragen en de doelstellingen van de bachelorproef.
- **Hoofdstuk 2** In dit hoofdstuk wordt de relevante literatuur rond dependency management, bestaande tools en oplossingen, technische schuld en AI-agentframeworks en workflow-automatisering geanalyseerd.
- **Hoofdstuk 3** Dit hoofdstuk bespreekt de onderzoeksaanpak, het ontwerp van de systeemarchitectuur, de selectie van platformen en de implementatie van de proof-of-concept.
- **Hoofdstuk 4** Dit hoofdstuk bevat de requirementsanalyse voor het AI-gestuurde dependencydashboard in de context van Springbok.
- **Hoofdstuk 5** Dit hoofdstuk licht het ontwerp en de implementatie toe van het proof-of-concept, inclusief de Mastra-agent en de n8n-workflow.
- **Hoofdstuk 6** In dit hoofdstuk worden de resultaten van de implementatie en experimenten gepresenteerd, en wordt de oplossing geëvalueerd op basis van de gedefinieerde vereisten en criteria.

- **Hoofdstuk 7** Dit hoofdstuk vat de belangrijkste bevindingen samen en formuleert aanbevelingen voor verdere uitwerking en mogelijke toekomstige verbeteringen.

2

Stand van zaken

Dit hoofdstuk schetst de stand van zaken rond dependencybeheer, technische schuld, bestaande automatiseringsoplossingen en recente ontwikkelingen in AI-agents en workflow-automatisering. Het doel is om stap voor stap te onderbouwen waarom dependency-updates vandaag een relevant onderhoudsprobleem vormen, waarom klassieke oplossingen slechts gedeeltelijk verlichting bieden en waarom er nog ruimte is voor een AI-ondersteunde aanpak. Op basis van die literatuur wordt uiteindelijk de onderzoeksniche van deze bachelorproef afgebakend.

2.1. Dependency-updates als onderhoudslast

Softwareprojecten steunen in toenemende mate op open-source dependencies, waardoor het onderhoud van externe bibliotheken een structureel onderdeel is geworden van moderne softwareontwikkeling. Die afhankelijkheid verhoogt de snelheid waarmee teams kunnen bouwen, maar maakt projecten tegelijk kwetsbaar voor verouderde of onveilige versies van componenten die buiten de eigen codebasis liggen (Pashchenko et al., [2018](#)). Dependencybeheer is daardoor niet langer een louter technische beheertaak, maar een voortdurende onderhoudsverantwoordelijkheid die rechtstreeks samenhangt met stabiliteit, veiligheid en evoleerbaarheid van software.

2.1.1. Kwetsbare en verouderde dependencies in de praktijk

Onderzoek toont aan dat kwetsbaarheden in externe libraries niet uitzonderlijk zijn en dat veel van die problemen in principe relatief eenvoudig op te lossen zijn door te upgraden naar een veiligere versie Pashchenko et al. ([2018](#)). Toch voeren ontwikkelteams dergelijke updates in de praktijk vaak niet consequent door. Dat wijst erop dat het probleem niet alleen in de aanwezigheid van kwetsbare dependencies zit, maar ook in de manier waarop teams de bijhorende informatie verwerken,

beoordelen en opvolgen. De uitdaging ligt dus niet enkel in detectie, maar vooral in het omzetten van update-informatie naar tijdige actie.

Daarnaast maakt het onderscheid tussen daadwerkelijk in productie gebruikte dependencies en packages die enkel in ontwikkeling of testing voorkomen duidelijk dat niet elke kwetsbaarheid dezelfde impact heeft Pashchenko et al. (2018). Juist daarom is context belangrijk: teams moeten kunnen inschatten welke dependency werkelijk risico oplevert en welke update effectief prioriteit verdient. Zonder dat onderscheid ontstaat al snel een overload aan signalen die moeilijk te vertalen is naar concrete onderhoudsbeslissingen.

2.1.2. Technische schuld als gevolg van uitgesteld onderhoud

Wanneer updates te lang worden uitgesteld, groeit de onderhoudslast uit tot technische schuld. Die schuld ontstaat wanneer korte termijnkeuzes, zoals het uitstellen van een update om tijd te besparen, op langere termijn leiden tot meer complexiteit, hogere kosten en grotere kans op fouten Behutiye et al. (2024) en Ruiz et al. (2024). In die zin is het uitblijven van dependency updates niet alleen een operationeel probleem, maar ook een bron van structurele accumulatie van onderhoudsrisico.

Empirisch werk toont aan dat technische schuld in industriële context concreet voelbaar wordt via verminderde productiviteit, vertragingen en een grotere foutkans bij softwarelevering Besker (2020). Voor dependencybeheer betekent dit dat elke gemiste update niet noodzakelijk meteen een incident veroorzaakt, maar wel bijdraagt aan een groeiende achterstand die latere wijzigingen moeilijker en duurder maakt. Het probleem is dus cumulatief: hoe langer teams wachten, hoe groter de inspanning wordt om de achterstand nog gecontroleerd in te halen.

2.1.3. De kloof tussen risico en actie

De kern van het probleem ligt niet in het ontbreken van updates, maar in de kloof tussen het signaleren van een risico en het effectief ondernemen van actie. Ontwikkelteams krijgen vaak wel informatie over nieuwe versies, securityfixes of breaking changes, maar die informatie is verspreid, technisch en tijdsintensief om te interpreteren. Daardoor wordt een update pas opgevolgd wanneer de noodzaak duidelijk zichtbaar wordt, terwijl het moment waarop een update het goedkoopst en veiligst is, vaak al voorbij is.

Die kloof verklaart waarom dependency updates vaak blijven liggen, zelfs wanneer de nood aan onderhoud rationeel duidelijk is. Teams moeten niet alleen weten dat er een update bestaat, maar ook wat de inhoud en impact van die update is. Precies daar ontstaat de link met de rest van dit hoofdstuk: bestaande oplossingen proberen die informatie deels te structureren, maar ze lossen het interpretatieprobleem niet volledig op.

2.2. Bestaande oplossingen en hun grenzen

Momenteel bestaan er reeds meerdere vormen van ondersteuning voor dependencybeheer, gaande van bots die updates detecteren tot tools die automatisch een pull request aanmaken. Die oplossingen verlagen de drempel om updates zichtbaar te maken, maar nemen het inhoudelijke beoordelingswerk nog niet weg. Daardoor verschuift de taak van detectie wel naar automatische systemen, maar blijft de interpretatie van changelogs, compatibiliteit en risico's grotendeels bij de ontwikkelaar liggen Erlenhov et al. (2022) en Pashchenko et al. (2018).

2.2.1. Dependencybots: wat ze wel en niet doen

Dependencybots zoals Dependabot zijn nuttig omdat ze repetitieve en goed afgebakende taken automatiseren. Ze signaleren nieuwe versies, openen pull requests en zorgen ervoor dat updates niet volledig over het hoofd worden gezien He et al. (2023). In die zin vormen ze een belangrijke stap in het verlagen van de operationele last rond afhankelijkheden, vooral omdat ze een groot deel van het handmatige opvolgwerk uit handen nemen He et al. (2023).

Tegelijk blijven deze bots beperkt in hun inhoudelijke bijdrage. Ze kunnen doorgaans aantonen dat er een update is, maar niet overtuigend uitleggen dat de implicaties ervan zijn. Voor ontwikkelaars blijft het dus nodig om changelogs, release notes en migratie-informatie zelf door te nemen om te bepalen of een update veilig, dringend of complex is. De literatuur suggereert daarmee dat automation op zichzelf niet volstaat wanneer de resterende taak vooral bestaat uit interpretatie en besluitvorming Erlenhov et al. (2022).

2.2.2. De ontbrekende schakel: context en beslissingsondersteuning

De echte meerwaarde lijkt te liggen in context: een systeem dat niet enkel meldt dat er iets veranderd is, maar ook helpt begrijpen wat die verandering betekent. Voor dependencybeheer is dat belangrijk omdat release notes vaak uit tekst, technische verwijzingen en impliciete aanames bestaan. Een ontwikkelaar moet die informatie vertalen naar de concrete context van het project, terwijl veel tools vandaag vooral focussen op de aanwezigheid van een update en niet op de inhoudelijke duiding ervan.

Daarmee ontstaat een duidelijk tekortkoming tussen detectie en beslissing. Bestaande oplossingen tonen aan dat updates bestaan en soms zelfs hoe dringend ze zijn, maar ze bieden zelden een compacte, contextbewuste samenvatting die het team helpt om snel te beslissen. Juist die vertaalslag van ruwe updates-informatie naar bruikbare beslissingsinformatie vormt een relevante onderzoeksvraag binnen dit domein.

2.2.3. Bijzondere uitdagingen bij private repositories en registries

De problematiek wordt complexer wanneer dependencies niet publiek beschikbaar zijn, maar via private repositories of interne repositories worden beheerd. In zulke omgevingen is automatische opvolging vaak lastiger, omdat tooling niet altijd dezelfde toegang en metadata krijgt als bij publieke packages. Daardoor kan update informatie minder volledig zijn of moeten teams alternatieve toegangsmechanismen voorzien om de nodige data überhaupt op te halen.

Dat maakt private ecosystemen extra relevant voor onderzoek naar dependencybeheer. Niet alleen is de hoeveelheid beschikbare informatie daar vaak beperkter, ook de operationele drempel om die informatie veilig te ontsluiten is hoger. Hierdoor worden de beperkingen van klassieke bots nog scherper zichtbaar: waar publieke packages vaak nog relatief goed ondersteund zijn, ontstaan in private omgevingen sneller blinde vlekken en manuele tussenstappen.

2.3. AI-agents en workflow automatisering

Wanneer de informatie over updates te verspreid of te contextafhankelijk wordt voor eenvoudige automatisering, komen AI-agents en workflow automatisering in beeld. Die technologieën zijn vooral interessant wanneer een taak bestaat uit meerdere stappen, waarbij informatie uit verschillende bronnen moet worden gecombineerd, geïnterpreteerd en teruggekoppeld naar een gebruiker of systeem Joshi (2025), Kim et al. (2024), en Zhu et al. (2025). In dat opzicht sluiten dependency updates goed aan bij het soort probleem waarvoor agentgebaseerde systemen ontworpen worden.

2.3.1. Geschiktheid van een taak voor een AI-agent

AI-agents zijn vooral nuttig in situaties waar een systeem die geen vaste handeling moet uitvoeren, maar iteratief informatie moet verzamelen, analyseren en samenvatten. Uit het onderzoek van Joshi (2025) en Zhu et al. (2025) blijkt dat agents bijzonder geschikt zijn voor taken met gestructureerde input, tekstuele context en meerdere beslismomenten. Dat maakt ze relevant voor softwarecontexten waarin logs, release notes, notificaties of documentatie samen moeten worden geïnterpreteerd.

Dependency analyse past goed in dat profiel. Een update bestaat vaak niet uit een duidelijk bericht, maar uit een combinatie van registry metadata, release notes, changelogs en soms aanvullende webdocumentatie. De taak is dus niet alleen informatief, maar ook selectief: niet elk detail is even relevant, en de bruikbare conclusie hangt af van de combinatie van bronnen. Daardoor lijkt dit domein geschikt voor een agent die informatie eerst verzamelt en vervolgens compact teruggeeft.

2.3.2. Agentic design: tools, orkestratie en feedbackloops

In de literatuur over agentarchitecturen keren telkens dezelfde bouwstenen terug: een taalmodel, tools, een vorm van orkestratie en een feedbacklus waarin eerdere resultaten worden meegenomen in volgende stappen Kim et al. (2024) en Zhu et al. (2025). Tools laten een agent toe om externe bronnen op te vragen of acties uit te voeren, terwijl orkestratie bepaalt in welke volgorde die stappen gebeuren. Feedbackloops maken het mogelijk om nieuwe informatie opnieuw te integreren wanneer de eerste bron onvoldoende blijkt.

Voor het type probleem dat in deze bachelorproef centraal staat, is dat relevant omdat dependencyinformatie vaak niet volledig in een bron zit. Een agent moet daarom kunnen beginnen bij een primaire bron en indien nodig overschakelen naar aanvullende context. De literatuur ondersteunt zo het idee dat agentgedrag niet noodzakelijk draait om autonomie als doel op zich, maar om gecontroleerde iteratie in informatieverzameling en interpretatie.

2.3.3. Code gedreven versus low-code orkestratie

De manier waarop zulke agentische processen worden georganiseerd, verschilt sterk tussen code gedreven en low-code benaderingen. Code gedreven systemen bieden doorgaans meer controle over de volgorde van stappen, de dataflow en de fouthandeling, terwijl low-code platformen nadruk leggen op visuele samenstelling, sneller configuratie en toegankelijkheid voor een breder profiel van gebruikers Wali et al. (2025). Beide benaderingen kunnen nuttig zijn, maar ze leggen andere accenten in termen van flexibiliteit, transparantie en onderhoudbaarheid.

Voor dependencybeheer is dat onderscheid relevant omdat de taak enerzijds voldoende gestructureerd is om in een workflow te vatten, maar anderzijds genoeg nuance bevat om baat te hebben bij expliciete controle over toolgebruik en bronselectie. De literatuur suggereert dus niet een enige juiste aanpak, maar wel dat de keuze tussen een code gedreven of low-code orkestratie mee bepaald wordt door de gewenste balans tussen controle en snelheid.

2.3.4. Tavily als zoek- en extractielaag

Naast de workflowtools en agentframeworks bestaan ook bouwstenen die specifiek gericht zijn op het ophalen en voorbereiden van externe webinformatie voor AI-toepassingen. Tavily is een zoekdienst die ontworpen is voor AI-agents en RAG-workflows en biedt onder meer een web search API, een extractie-API en mogelijkheden voor site crawling en domeinfiltering Tavily (2026c, 2026d). In deze bachelorproef is Tavily relevant omdat de tool niet enkel zoekresultaten teruggeeft, maar deze ook structureert op een manier die beter aansluit bij gebruik door taalmodellen, bijvoorbeeld via titels, URL's, samenvattingen en scoringsinformatie Tavily (2026d) en Tavily AI (2025).

De keuze voor Tavily wordt vooral ingegeven door vier criteria. Ten eerste sluit de

tool goed aan bij het beoogde gebruik in een AI-gestuurde analyseketen, omdat ze expliciet ontworpen is voor agentische zoekscenario's en daardoor beter past dan een algemene zoekmachine. Ten tweede ondersteunt Tavily gerichte filtering op domein en inhoud, wat belangrijk is om officiële documentatie, release notes en changelogs te prioriteren boven minder betrouwbare bronnen Tavily (2026b, 2026d). Ten derde levert de tool output die rechtstreeks bruikbaar is voor verdere tekstverwerking, waardoor minder nabewerking nodig is dan bij klassieke zoekresultaten. Ten vierde past de tool binnen een workflow waarin actuele informatie snel moet worden opgehaald zonder dat de agent zelf uitgebreide scraping- of extractielogica moet bevatten.

Voor toepassingen zoals dependency-analyse is dat relevant omdat release notes, migratiegidsen en changelogs niet altijd rechtstreeks beschikbaar zijn via gestructureerde registries. In zulke gevallen kan een zoeklaag zoals Tavily helpen om recente en contextuele informatie op te halen uit officiële documentatie of andere relevante webbronnen. De tool ondersteunt bovendien een meer gecontroleerde bronselectie dan een generieke zoekmachine, waardoor de kans kleiner wordt dat irrelevante of verouderde informatie in de analyse terechtkomt Tavily (2026b, 2026d). Daardoor vormt Tavily een nuttige schakel wanneer een agent of workflow verder moet zoeken dan de primaire package-metadata of repository-informatie.

Binnen een geautomatiseerde analyseketen kan Tavily dus functioneren als aanvullende informatiebron tussen de ruwe dependencygegevens en de uiteindelijke AI-samenvatting. In vergelijking met alternatieven zoals een klassieke zoekmachine of zelf opgebouwde scrapinglogica biedt Tavily in deze context vooral meer doelgerichtheid, betere structuur van de output en minder implementatiecomplexiteit. De literatuur en documentatie suggereren daarom dat dit type zoekdienst vooral waardevol is wanneer het systeem actuele, brongebonden informatie nodig heeft die nog niet in de primaire databron aanwezig is Tavily (2026a, 2026c). In die zin vult Tavily de kloof tussen eenvoudige metadata-opvraging en diepere tekstuele interpretatie van online bronnen.

2.4. Platformen en ontwerpkeuzes

Binnen die categorieën worden onder meer Mastra AI, n8n en de agent- en toolgebaseerde configuratiemogelijkheden van grote taalmodelplatformen als representatieve opties besproken¹

De keuze om deze platformcategorieën eerst breed te schetsen is belangrijk omdat zij elk een andere ontwerpfilosofie vertegenwoordigen. Code-gedreven frameworks bieden maximale controle en uitbreidbaarheid, workflowtools focussen op

¹Zie de officiële documentatie en platformpagina's van Mastra (<https://github.com/mastra-ai/mastra>, <https://mastra.ai/docs/observability/overview>), n8n (<https://docs.n8n.io>) en OpenAI (<https://developers.openai.com/api/docs/guides/function-calling>, <https://developers.openai.com/api/docs/guides/tools>, <https://developers.openai.com/api/docs/guides/agents>).

visuele orkestratie en snelle integratie, en API-gestuurde configuraties laten toe om agentgedrag direct te sturen vanuit een model- en toolset. Door deze categorieën naast elkaar te plaatsen, wordt duidelijk welke afwegingen spelen bij de selectie van een geschikte aanpak voor dependencybeheer.

2.4.1. Code-gedreven frameworks

Code-gedreven frameworks zoals Mastra AI zijn interessant wanneer de ontwikkelaar maximale controle wil over de agentlogica, de tools en de integratie met een bestaande codebase. De literatuur rond agentframeworks benadrukt dat zulke systemen vooral waardevol zijn wanneer uitbreidbaarheid belangrijk is Devon Sun (2025) en MastraAI (2025, 2026). Hun sterkte ligt in technische precisie en in de mogelijkheid om stappen, tools en antwoorden nauwkeurig te structureren.

Dat maakt dit type platform aantrekkelijk voor taken waarbij de kwaliteit van de output niet alleen afhangt van het taalmodel, maar ook van de manier waarop inputbronnen worden gekozen en gecombineerd. Voor dependencybeheer is dat relevant omdat updates inhoudelijk moeten worden samengevat zonder essentiële details te verliezen. Code-gedreven benaderingen bieden in dat geval de mogelijkheid om de tussenstappen expliciet te modelleren.

2.4.2. Low-code workflowtools

Low-code workflowtools zoals n8n richten zich op het visueel verbinden van data-bronnen, triggers en verwerkingsstappen. De literatuur benadrukt dat zulke tools vooral goed passen bij repetitieve, eventgedreven processen waarin extreme systemen aan elkaar moeten worden gekoppeld Hatchworks (2025), n8n (2025), en Proser (2025). Hun waarde ligt minder in diepgaande programmatische controle en meer in snelle samenstelling en duidelijke visualisatie van de processtroom.

Voor AI-ondersteunde dependencyanalyse betekent dit dat low-code tools goed bruikbaar zijn wanneer de nadruk ligt op integratie en automatisering van bestaande stappen. Ze maken het eenvoudiger om informatie uit verschillende bronnen samen te brengen en door te geven aan een model, maar bieden minder nuance dan een volledige code-gedreven aanpak wanneer de logica complexer wordt. De literatuur toont daarmee aan dat low-code vooral sterk is in orkestratie, maar niet noodzakelijk in nauwkeurige agentcontrole.

2.4.3. API-gestuurde agentconfiguratie

Een derde optie is een meer directe, API-gestuurde aanpak waarbij een taalmodel via function calling of tool calling zelf stappen kan initiëren binnen een gecontroleerde loop OpenAI (2026a, 2026b, 2026c). In die benadering wordt de agent niet noodzakelijk ingebed in een zwaar framework, maar geconfigureerd via een model, instructies en expliciet gedefinieerde tools. Dat levert een compacte maar krachtige manier op om iteratieve informatieverwerking te organiseren.

Deze aanpak is vooral interessant wanneer de ontwikkelaar de agentflow nauw wil afstemmen op de taak, maar geen extra orkestratielaag wil toevoegen die de controle overneemt. De literatuur wijst erop dat dergelijke API-gestuurde agentconfiguraties nuttig zijn voor taken waarin modellen meerdere tools achter elkaar moeten kunnen aanroepen en de resultaten daarvan in een gestructureerde output moeten samenbrengen OpenAI (2026b, 2026c). Daardoor vormen ze een relevante ontwerpoptie binnen het bredere landschap van AI-automatisering.

2.4.4. Vergelijkend overzicht van platformcategorieën

Tabel 2.1 vat de besproken platformcategorieën samen aan de hand van hun voor- naamste kenmerken en representatieve voorbeelden. Dit overzicht onderbouwt de uiteindelijke platformkeuze die in de volgende sectie wordt toegelicht.

Tabel 2.1: Vergelijkend overzicht van platformcategorieën voor AI-agenten (gebaseerd op Anthropic, 2025; Langflow, 2025; Nair, 2025; OpenAI, 2025; Turing, 2026)

Categorie	Kenmerken	Voorbeelden
Code-gedreven frameworks	Hoge controle over agent-logica, tools en integratie; geschikt voor uitbreidbare en technisch nauwkeurige implementaties.	Mastra AI, LangGraph, CrewAI, AutoGen, Semantic Kernel, LlamaIndex
Low-code workflowtools	Visuele orkestratie van triggers, databronnen en verwerkingsstappen; sterk in snelle integratie en procesautomatisering.	n8n, Flowise, LangFlow, Dify, Gumloop
API-gestuurde configuraties	Directe aansturing via model, instructies en tool/function calling; compact en flexibel voor gecontroleerde agentflows.	OpenAI Agent Builder, OpenAI Agents SDK, Anthropic Tool Use, Google ADK, Function Calling / Tool Calling APIs

2.5. Synthese: de onderzoeksniche

De literatuur maakt duidelijk dat dependency updates een reële onderhoudslast vormen. Kwetsbare en verouderde dependencies verhogen het risico op technische schuld, terwijl het uitstellen van updates de onderhoudsachterstand verder doet groeien Behutiye et al. (2024), Besker (2020), Pashchenko et al. (2018), en Ruiz et al. (2024). Tegelijk tonen bestaande oplossingen aan dat detectie op zichzelf niet voldoende is: dependencybots maken updates zichtbaar, maar laten de inhoud-

lijke interpretatie grotendeels aan ontwikkelaars over Erlenhov et al. (2022). Daarmee ontstaat een duidelijke tekortkoming. Er is nood aan ondersteuning die niet enkel meldt dat er een update bestaat, maar ook helpt om de betekenis van die update snel te begrijpen binnen de context van een concreet project. AI agents en workflowautomatisering bieden hiervoor een interessant antwoord, omdat ze informatie uit meerdere bronnen kunnen combineren en samenvatten Joshi (2025), Kim et al. (2024), Wali et al. (2025), en Zhu et al. (2025). De literatuur wijst dus op een volgende stap tussen eenvoudige detectietools en volledig handmatige evaluatie. De combinatie die in deze bachelorproef onderzocht wordt, situeert zich precies in die ruimte: een AI ondersteunde laag bovenop dependencyopvolging die changelog- en update-informatie niet alleen verzamelt, maar ook bruikbaar tracht te maken voor beslissingsvorming. Daarmee wordt beoogd om de kloof tussen risicosignalering en actie te verkleinen. Daarmee wordt beoogd om de kloof tussen risicosignalering en actie te verkleinen. De relevante onderzoeksniche ligt dus niet in het louter detecteren van updates, maar in het ondersteunen van de interpretatie ervan zodat teams sneller en gericht kunnen beslissen over onderhoud.

3

Methodologie

Dit onderzoek volgt de opzet van een vergelijkende studie met proof-of-concept, aangevuld met literatuuronderzoek en een requirements-analyse binnen een concrete bedrijfscontext. Het doel is om enerzijds het probleem en de oplossingsruimte rond dependencybeheer in kaart te brengen, en anderzijds een apart dashboard-prototype te bouwen en technisch te evalueren binnen een realistische ontwikkelomgeving.

3.1. Fase 1 - Verdiepende literatuurstudie en probleemverkenning

In de eerste fase van de bachelorproef wordt de probleemcontext verder uitgediept aan de hand van een systematische literatuurstudie rond dependency management, kwetsbare dependencies, technische schuld en bestaande dependencybots. Daarnaast wordt vakliteratuur over AI-agents en workflow-automatisering bestudeerd om na te gaan welke concepten en benaderingen relevant zijn voor een AI-gestuurd dependency-dashboard.

Tijdens deze fase worden ook één of enkele representatieve projecten van Springbok geselecteerd en geanalyseerd (projectstructuur, gebruikte technologieën, dependency-ecosystemen, huidige processen rond updates). De belangrijkste deliverables zijn:

- een uitgewerkte en geactualiseerde literatuurstudie;
- een scherpere probleemomschrijving toegespitst op de gekozen bedrijfscontext en projecten;
- een eerste set geïnformeerde hypothesen over de verwachte impact van een AI-gestuurd dependency-dashboard.

3.2. Fase 2 - Requirements-analyse en architectuurontwerp

In de tweede fase wordt eerst een gerichte requirements-analyse uitgevoerd in samenwerking met de co-promotor via interviews en workshops worden functionele en niet-functionele eisen expliciet gemaakt, zoals:

- functionele eisen: integratie met GitHub, beheer van projecten en dependencies in het dashboard, detectie van nieuwe versies via publieke registries, analyse en samenvatting van changelogs en release notes, aanduiding van kritici- teit (security, bugfix, feature), prioritering van updates;
- niet-functionele eisen: schaalbaarheid, betrouwbaarheid, traceerbaarheid (logging en audittrail), gebruiksvriendelijkheid en onderhoudbaarheid.

Op basis van deze requirements wordt vervolgens een systeemarchitectuur ontworpen voor de webapplicatie en de from-scratch ontworpen AI-agent. De architectuur beschrijft onder meer de globale componenten (frontend, backend, databank, integraties met GitHub en registries), de verantwoordelijkheden per component, de datastromen (bijvoorbeeld van dependency-informatie naar AI-analyse en terug naar het dashboard) en de plaats van eventueel ondersteunende workflow- of agentplatformen. Deliverables van fase 2 zijn:

- een requirementsdocument met duidelijke prioritering;
- een architectuurbeschrijving (diagrammen, componentbeschrijvingen, data-modellen);
- een lijst van geselecteerde technologieën en tools, gemotiveerd vanuit de requirements.

3.3. Fase 3 - Implementatie van het proof-of-concept

In fase 3 wordt de ontworpen architectuur omgezet in een werkend proof-of-concept. De kern bestaat uit een afzonderlijke webapplicatie waarin per project dependencies worden bijgehouden en verrijkt met informatie uit externe bronnen, zoals package registries en repositorygegevens. De applicatie wordt opgezet in een realistische context, zodat de proof-of-concept kan worden getest op basis van echte project-gegevens en representatieve update-informatie.

Parallel wordt een AI-ondersteunde analysekern geïntegreerd die changelogs, release notes en andere relevante update-informatie verwerkt en omzet naar compacte samenvattingen en indicaties van de aard van een update. In deze fase worden meerdere varianten van die AI-integratie uitgewerkt, zodat later een gerichte vergelijking kan worden gemaakt tussen verschillende vormen van agent- en workflow-orkestratie. De frontend visualiseert vervolgens de verzamelde dependencygegevens, de AI-gegenereerde toelichting en de voorgestelde prioritering. Deliverables van fase 3 zijn:

- een werkend prototype van het dashboard met integratie naar project- en dependencygegevens;
- meerdere geïmplementeerde varianten van de AI-ondersteunde analysekern;
- technische documentatie over de implementatie, architectuur en integratiepunten.

3.4. Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak

De vierde fase richt zich op het onderbouwen en verfijnen van de gemaakte ontwerpkeuzes aan de hand van een gerichte vergelijking van de drie uitgewerkte AI-agentoplossingen en eventuele architecturale varianten. Op basis van de requirements uit fase 2 wordt een set *eenvoudig meetbare* evaluatiecriteria opgesteld, zoals:

- gemiddelde verwerkingstijd per update (doorlooptijd van input tot samenvatting);
- technische inspanning voor opzet en onderhoud (bijvoorbeeld aantal configuratiestappen, hoeveelheid code);
- kwaliteit van samenvattingen gemeten via een eenvoudige beoordelingsschaal door de co-promotor (bijvoorbeeld 1–5 voor duidelijkheid en relevantie);
- fout- of faalratio (bijvoorbeeld aantal mislukte runs of onvolledige resultaten per tien updates).

Aan de hand van een aantal representatieve scenario's (bijvoorbeeld updates met alleen kleine bugfixes, updates met beveiligingspatches, updates met potentieel brekende wijzigingen) worden OpenAI Agent Builder, n8n en Mastra AI onder dezelfde omstandigheden getest en met elkaar vergeleken. De focus ligt hierbij niet op een brede marktanalyse, maar op het versterken of bijsturen van de gekozen architectuur en het bepalen welke aanpak het meest geschikt is als basis voor het verdere gebruik van het dashboard in deze specifieke use case. Deliverables van fase 4 zijn:

- experimentele beschrijvingen van de uitgeteste varianten;
- meetresultaten volgens de gedefinieerde criteria (doorlooptijd, inspanning, beoordelingsscores, foutpercentages);
- een gemotiveerde keuze of bevestiging van de uiteindelijke architectuur en AI-configuratie.

3.5. Fase 5 - Evaluatie en rapportering

In de laatste fase wordt het proof-of-concept geëvalueerd op basis van de bevindingen uit de implementatie en de vergelijking van de verschillende varianten. Daarbij wordt gekeken naar de bruikbaarheid van het dashboard, de kwaliteit van de AI-gegenereerde samenvattingen, de technische haalbaarheid van de oplossing en de mate waarin het systeem aansluit bij de vooropgestelde vereisten. Waar relevant wordt deze evaluatie aangevuld met feedback van de co-promotor of andere betrokkenen uit de bedrijfscontext.

De bevindingen uit deze evaluatie worden samengebracht om de onderzoeksvragen te beantwoorden en de ontwerpkeuzes te reflecteren. Deze fase mondt uit in de finale rapportering van de bachelorproef, waarin probleemstelling, methode, implementatie, resultaten en conclusies in samenhang worden gepresenteerd. Deliverables van fase 5 zijn:

- een synthese van de evaluatiebevindingen en aandachtspunten;
- de uitgewerkte bachelorproef met geïntegreerde beschrijving van probleem, methode, resultaten en conclusies;
- aanbevelingen voor verdere ontwikkeling of vervolgonderzoek.

De globale fasering en tijdsplanning van het onderzoek wordt weergegeven in Figuur 3.1.



4

Requirementsanalyse

Deze requirementsanalyse beschrijft de functionele en niet-functionele vereisten van DepGuard AI, de AI-gestuurde dependencydashboardoplossing die in deze bachelorproef wordt ontwikkeld. De analyse bouwt voort op de probleemstelling, literatuurstudie en het overleg met de co-promotor. Het platform wordt opgezet als een losstaande webapplicatie, onafhankelijk van bestaande tools zoals Dependabot, en richt zich op het samenvatten van release notes, het beoordelen van de criticiteit van updates en het ondersteunen van het kernteam bij het prioriteren van dependency updates.

4.1. Stakeholders

De belangrijkste stakeholders van DepGuard AI zijn:

- **Kernteam voor platform- en dependencybeheer:** primaire gebruikers, verantwoordelijk voor het opvolgen en doorvoeren van dependency-updates en dagelijks gebruik van het dashboard voor overzicht, prioritering en AI samenvattingen.
- **Developers binnen projecten:** secundaire gebruikers die aan individuele projecten werken, update-adviezen raadplegen en projectleden en dependencies beheren.
- **Co-promotor / bedrijfsbegeleider:** opdrachtgever vanuit Springbok, die de bruikbaarheid en kwaliteit van de oplossing evalueert en mee de requirements scherpstelt.
- **Promotor (HOGENT):** academische begeleider die de methodologie, uitvoering en kwaliteit van het eindwerk beoordeelt.

- **Externe platforms:** GitHub en de npm-registry leveren project- en versiemetadata via hun APIs en vormen zo cruciale bronnen voor het systeem.

4.2. Functionele vereisten

De functionele vereisten zijn gegroepeerd per domein en geprioriteerd volgens de MoSCoW-methode (Must Have, Should Have, Won't Have voor dit proof-of-concept).

4.2.1. Authenticatie en gebruikersbeheer

De volgende functionele vereisten gelden voor authenticatie en gebruikersbeheer.

- **F-01 (Must Have):** Gebruikersregistratie en login via e-mail en wachtwoord (Supabase Auth).
- **F-02 (Must Have):** Aanmaken van een bedrijfsaccount (multi-tenant).
- **F-03 (Must Have):** Uitnodigen van teamleden via e-mail met rolkoppeling (Resend API).
- **F-04 (Must Have):** Rollen per bedrijf: owner, project_lead, developer, tester (RBAC).
- **F-05 (Must Have):** Rollen per project: project_lead, developer (projectniveau RBAC).
- **F-06 (Should Have):** Uitnodiging accepteren via directe link zonder e-mail (fallbackflow).
- **F-07 (Could Have):** SSO / OAuth login (bijvoorbeeld GitHub of Google via Supabase OAuth).

4.2.2. Projectbeheer

De volgende functionele vereisten gelden voor projectbeheer.

- **F-08 (Must Have):** Aanmaken, bewerken en verwijderen van projecten (CRUD).
- **F-09 (Must Have):** Koppelen van een publieke GitHub-repository aan een project (GitHub-integratie).
- **F-10 (Must Have):** Uploaden van een `package.json` om dependencies in te laden.
- **F-11 (Must Have):** Toewijzen van teamleden aan projecten met projectrol.
- **F-12 (Must Have):** Overzicht van alle projecten per bedrijf met statusindicator.
- **F-13 (Could Have):** Archiveren of deactiveren van projecten.

4.2.3. Dependency scanning en versiebeheer

De volgende functionele vereisten gelden voor dependency scanning en versiebeheer.

- **F-14 (Must Have):** Automatisch detecteren van dependencies via GitHub (uit-lezen van `package.json` via de GitHub API).
- **F-15 (Must Have):** Ophalen van de meest recente versie via de npm-registry (`npmjs.org`).
- **F-16 (Must Have):** Vergelijken van huidige versie met meest recente versie (semver-vergelijking).
- **F-17 (Must Have):** Classificeren van updates als major, minor of patch (semver-classificatie).
- **F-18 (Must Have):** Opslaan van dependency-informatie (naam, versies, status) in Supabase Postgres.
- **F-19 (Should Have):** Manueel hertriggeren van een scan door de gebruiker.
- **F-20 (Could Have):** Automatisch periodiek scannen via een geplande job (cron).
- **F-21 (Won't Have):** Ondersteuning voor meerdere registries (bijv. PyPI, Maven).

4.2.4. AI-analyse en samenvatting

De volgende functionele vereisten gelden voor AI-analyse en samenvatting.

- **F-22 (Must Have):** AI-agent analyseert changelogs en release notes van npm en GitHub (Mastra-agent met LLM-backend).
- **F-23 (Must Have):** Genereren van een begrijpelijke samenvatting per dependency-update.
- **F-24 (Must Have):** Classificeren van het updatetype: security fix, bugfix, nieuwe feature, breaking change.
- **F-25 (Must Have):** Aanduiden of een update veilig, risicovol of kritiek lijkt (risico-indicatie).
- **F-26 (Must Have):** Opslaan van de AI-samenvatting per dependency in het veld `ai_summary`.
- **F-27 (Must Have):** Ondersteuning voor meerdere AI-providers (OpenAI, Google Gemini).

- **F-28 (Should Have):** Alternatieve AI-workflow via n8n (low-code orkestratie als vergelijking met de Mastra-agent).
- **F-28b (Should Have):** Alternatieve AI-agent via OpenAI Agent Builder, waarbij een gehoste agent met tools wordt geconfigureerd als tweede alternatief naast Mastra en n8n.
- **F-29 (Should Have):** Verrijken van AI-input met webcontext via Tavily.
- **F-30 (Should Have):** Hertriggere van AI-analyse op verzoek van de gebruiker.
- **F-31 (Could Have):** Gestructureerde output met migratiestappen en deprecation-warnings.

4.2.5. Dashboard en visualisatie

De volgende functionele vereisten gelden voor het dashboard en visualisatie.

- **F-32 (Must Have):** Overzichtsdashboard met statistieken (verouderde packages, security-issues, recente activiteit).
- **F-33 (Must Have):** Detailweergave per project met lijst van dependencies en hun status.
- **F-34 (Must Have):** Weergave van de AI-samenvatting per dependency in het dashboard.
- **F-35 (Must Have):** Visuele badge of kleurcodering voor major/minor/patch en risico-niveau.
- **F-36 (Should Have):** Filteren en sorteren van dependencies op status, updatetype en criticiteit.
- **F-37 (Could Have):** Exporteren van een dependency-rapport als PDF of CSV.

4.2.6. Notificaties en communicatie

De volgende functionele vereisten gelden voor notificaties en communicatie.

- **F-38 (Must Have):** E-mailnotificatie bij uitnodiging van een nieuw teamlid (Resend API).
- **F-39 (Should Have):** E-mailnotificatie bij detectie van een kritieke security-update.
- **F-40 (Could Have):** In-app notificatie of badge bij nieuwe updates per project.
- **F-41 (Won't Have):** Integratie met Slack of Microsoft Teams voor updateberichten.

4.2.7. Regels en beslissingsondersteuning

De volgende functionele vereisten gelden voor regels en beslissingsondersteuning.

- **F-42 (Must Have):** AI geeft een advies per dependency: “veilig te updaten”, “voorzichtigheid geboden” of “kritiek”.
- **F-43 (Should Have):** Mogelijkheid om een dependency handmatig als “genegeerd” of “gedaan” te markeren.
- **F-44 (Could Have):** Definieren van updatebeleid per project (bijvoorbeeld patch-updates automatisch goedkeuren).
- **F-45 (Won't Have):** Automatisch aanmaken van pull requests of GitHub-issues op basis van advies.

4.3. Niet-functionele vereisten

De niet-functionele vereisten zijn afgestemd op de gekozen technische stack (Next.js, Supabase, Mastra, n8n, OpenAI-agent builder en externe LLms) en de context van Springbok.

4.3.1. Beveiliging

- **NFR-01:** Row Level Security (RLS) in Supabase zorgt ervoor dat gebruikers uitsluitend data zien van het eigen bedrijf en de toegewezen projecten.
- **NFR-02:** Authenticatie is verplicht voor alle API-routes en dashboardpagina's via Next.js-middleware.
- **NFR-03:** API-sleutels (OpenAI, Google, Resend, GitHub) worden uitsluitend als omgevingsvariabelen opgeslagen en zijn nooit zichtbaar in de frontend.
- **NFR-04:** Uithodigingstokens zijn eenmalig geldig en vervallen na acceptatie.

4.3.2. Prestaties

- **NFR-05:** De end-to-end AI-analyse (van dependency-input tot opgeslagen samenvatting) wordt onder normale omstandigheden binnen ongeveer 30 seconden afgerond.
- **NFR-06:** Paginalaadtijden voor het dashboard blijven onder 2 seconden bij een dataset tot ongeveer 200 dependencies per bedrijf.
- **NFR-07:** De n8n-workflow en de Mastra-agent worden parallel geëvalueerd op doorlooptijd; de resultaten worden vastgelegd in de evaluatiefase.

4.3.3. Schaalbaarheid

- **NFR-08:** De multi-tenant architectuur laat toe om meerdere bedrijven, projecten en gebruikers onafhankelijk van elkaar te beheren.
- **NFR-09:** Opslag op Supabase (PostgreSQL) ondersteunt schaalbaarheid via managed infrastructuur en connection pooling.
- **NFR-10:** De AI-agentlaag is modulair en kan worden uitgebreid met extra tools, registries of LLM-providers zonder ingrijpende wijzigingen aan de webapplicatie.

4.3.4. Betrouwbaarheid en foutafhandeling

- **NFR-11:** Bij het ontbreken van release notes in GitHub wordt teruggevallen op bestanden zoals `CHANGELOG.md`, `CHANGES.md` of `HISTORY.md`.
- **NFR-12:** Als er geen changelog-informatie beschikbaar is, vermeldt de AI-agent dit expliciet in de samenvatting.
- **NFR-13:** API-fouten (GitHub, npm, LLM) worden gelogd en weergegeven als leesbare foutmeldingen in de UI.
- **NFR-14:** De Mastra-server en de Next.js-applicatie zijn onafhankelijk startbaar; bij uitval van Mastra blijft het dashboard bruikbaar zonder AI-analyse.

4.3.5. Onderhoudbaarheid

- **NFR-15:** De volledige codebase is getypeerd in TypeScript.
- **NFR-16:** Logica is opgesplitst per verantwoordelijkheid: AI-agentcode, API-routes en UI-componenten zijn in aparte modules gestructureerd.

4.3.6. Bruikbaarheid

- **NFR-17:** Het dashboard is responsive en bruikbaar op verschillende schermformaten.
- **NFR-18:** AI-samenvattingen zijn geschreven in begrijpelijke taal, gericht op developers zonder diepgaande AI-kennis.
- **NFR-19:** Statusbadges voor updatetype en risiconiveau zijn kleurgecodeerd en direct interpreteerbaar.
- **NFR-20:** Het onboardingproces (registratie, bedrijf aanmaken, project koppelen en eerste scan) kan in minder dan vijf minuten worden doorlopen.

4.3.7. Traceerbaarheid en audit

- **NFR-21:** Mastra voorziet tracing en logging van agentbeslissingen, tool-calls en modelinteracties, zodat het gedrag van de AI-agent kan worden geanalyseerd.
- **NFR-22:** Alle AI-samenvattingen worden met tijdstempel opgeslagen zodat de evolutie van adviezen per dependency navolgbaar is.
- **NFR-23:** Gebruikersacties zoals scannen, analyseren en uitnodigen zijn via Supabase auditeerbaar.

4.3.8. Portabiliteit en installatie

- **NFR-24:** De applicatie kan lokaal worden gestart via `npm run dev` (Next.js) en `npm run dev:mastra` (Mastra) na configuratie van de omgevingsvariabelen.
- **NFR-25:** De database-opzet is volledig scriptbaar en vereist geen manuele tabellaanmaak.
- **NFR-26:** De applicatie is inzetbaar op courante cloudplatformen, bijvoorbeeld Vercel voor Next.js en Supabase Cloud voor de databank.

4.4. Samenvattende MoSCoW-prioritering

In totaal worden 26 Must Have-, 9 Should Have-, 8 Could Have- en 4 Won't Have-vereisten onderscheiden. De Must Have-vereisten dekken de kernfunctionaliteit van DepGuard Ai (authenticatie, projectbeheer, scanning, AI-analyse, dashboard en basisadvies), terwijl de overige categorieën extra bruikbaarheid, gemak of toekomstige uitbreidingen beschrijven.

5

Ontwerp en implementatie van het AI-gestuurde dashboard

In dit hoofdstuk wordt beschreven hoe de concepten uit de literatuur en de requirements analyse vertaald werden naar een werkende proof-of-concept. Eerst wordt de globale architectuur van het systeem toegelicht, gevolgd door de drie uitgewerkte AI-integraties en tot slot de koppeling met de webapplicatie. Op die manier vormt dit hoofdstuk de brug tussen het theoretische kader en de concrete realisatie van DepGuard AI.

5.1. Architectuur van het systeem

Het proof-of-concept voor DepGuard AI is opgezet als een moderne webarchitectuur met een duidelijke scheiding tussen frontend, backend, databank en AI-laag. De technologiekeuzes werden gemaakt op basis van de vereisten uit Fase 2 en de volgende criteria:

- **Frontend (Next.js,¹ Tailwind CSS,² shadcn/ui³):** Next.js biedt een robuuste full-stack basis met server-side rendering en API-routes. Bovendien is het een breed geadopteerde en mature stack binnen het React-ecosysteem, wat de beschikbaarheid van documentatie, voorbeelden en community-ondersteuning versterkt. Tailwind CSS en shadcn/ui zorgen voor consistente, toegankelijke componenten met minimale boilerplate.
- **Backend en databank (Supabase⁴):** PostgreSQL met Row Level Security voldoet aan de eisen voor multi-tenant isolatie, realtime updates en eenvoudige

¹<https://nextjs.org/docs>

²<https://tailwindcss.com/docs>

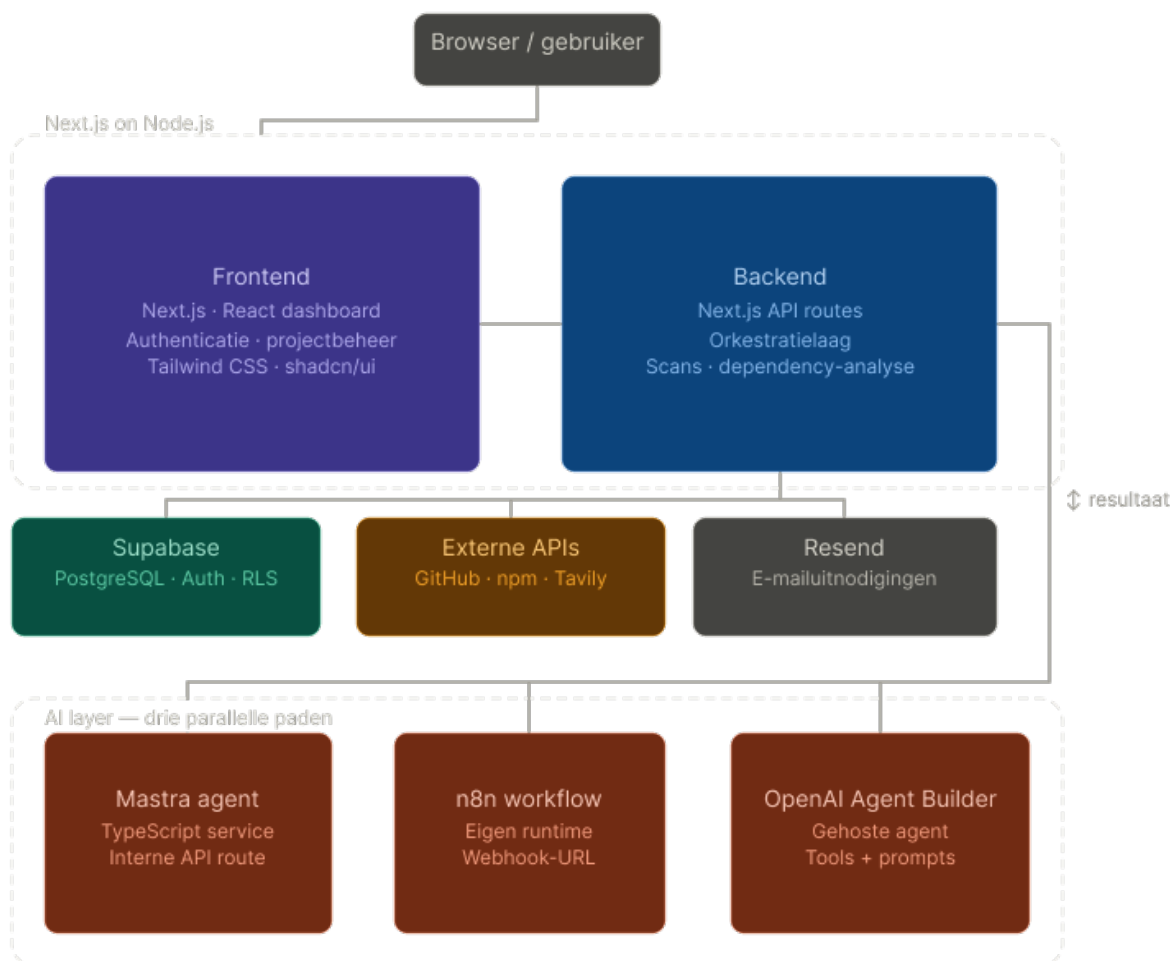
³<https://ui.shadcn.com/docs>

⁴<https://supabase.com/docs>

integratie met Next.js.

- **AI-kern (Mastra,⁵ n8n,⁶ OpenAI Agent Builder⁷):** Deze drie platformen vertegenwoordigen respectievelijk code-gedreven frameworks, low-code workflows en API-gestuurde configuraties. De selectie is gebaseerd op representativiteit, tool-ondersteuning, integratiegemak en documentatiekwaliteit.

Figuur 5.1 toont de globale systeemarchitectuur.



Figuur 5.1: Globale architectuur van DepGuard AI met frontend, backend, databank en AI-laag

De frontend is een Next.js dashboard dat authenticatie, projectbeheer en visualisatie van dependency- en AI-informatie verzorgt. Gebruikers kunnen bedrijven en projecten aanmaken, GitHub-repositories koppelen, dependency-scans opstarten en de door de AI gegenereerde samenvattingen en adviezen raadplegen. Voor de UI wordt gebruikgemaakt van Tailwind CSS en shadcn/ui, zodat componenten consistent en herbruikbaar zijn.

⁵<https://mastra.ai/docs>

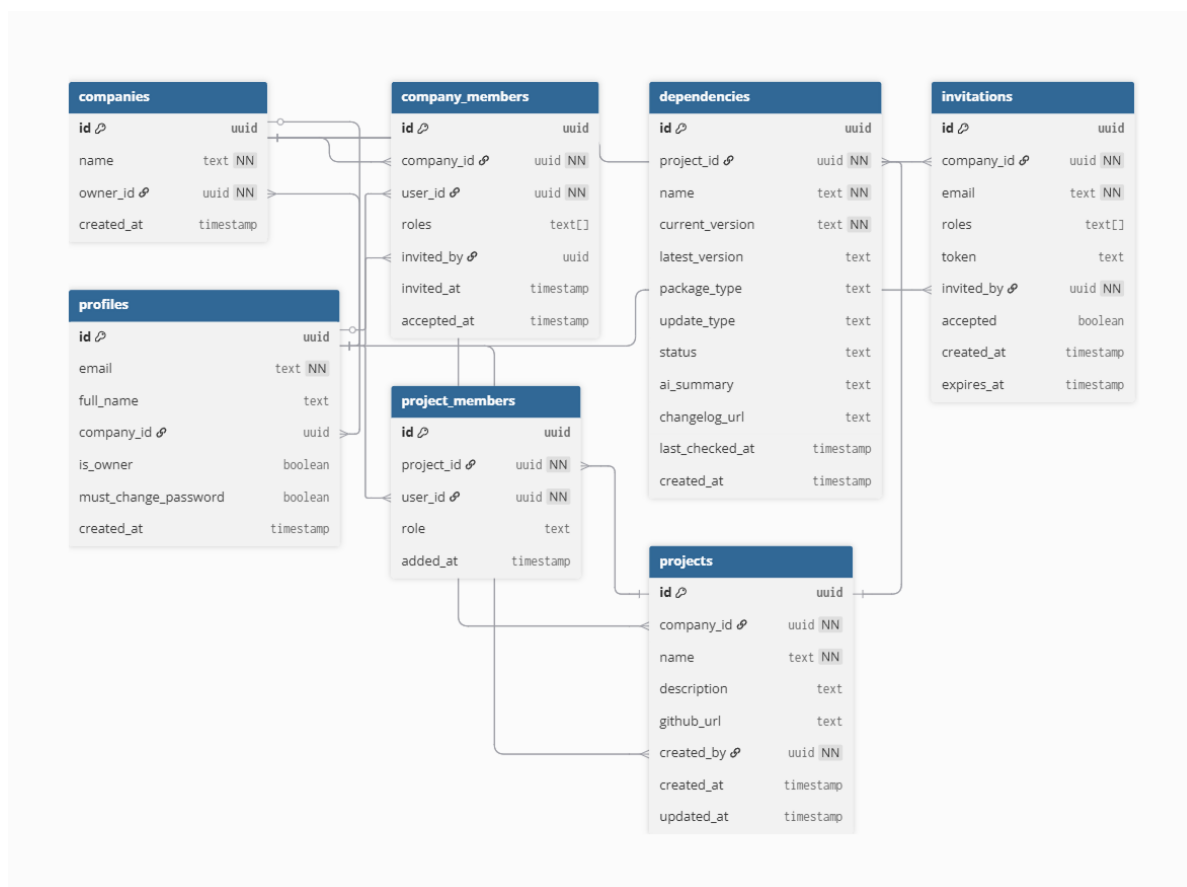
⁶<https://docs.n8n.io>

⁷<https://platform.openai.com/docs/guides/agents>

De backend bestaat uit een set Next.js API-routes die fungeren als orkestratielaag tussen de frontend, Supabase, externe APIs (GitHub, npm, Tavily) en de verschillende AI-componenten. Bij een scan leest de backend eerst de dependencyinformatie van een project (bijvoorbeeld uit `package.json` in GitHub), slaat de resultaten op in Supabase en roept vervolgens één van de AI-varianten aan: de mastra-agent, de n8n-workflow of de OpenAI-agent. De teruggestuurde samenvatting wordt gekoppeld aan de dependency en via het dashboard zichtbaar gemaakt voor de gebruiker.

Supabase verzorgt de persistente opslag van bedrijven, bedrijfleden, gebruikers, uitnodigingen, projecten, projectleden en dependencies, met Row Level Security om multi-tenant isolatie te garanderen. Het conceptuele datamodel is opgebouwd rond een centrale bedrijfsstructuur, waarvan gebruikers, projecten en dependencygegevens gekoppeld worden. De tabel `companies` bevat de kerngegevens van een organisatie en verwijst naar de eigenaar via `owner_id`. Via `company_members` worden bijkomende gebruikers aan een bedrijf gekoppeld, inclusief hun rol en het moment waarop ze werden uitgenodigd of geaccepteerd. De tabel `profiles` bewaart de profielgegevens van gebruikers en kan optioneel gekoppeld zijn aan een bedrijf.

De projectstructuur vertrekt vanuit `projects`, waar elk project behoort tot een bedrijf en een aanmaker heeft. Via `project_members` worden gebruikers aan projecten gekoppeld met een specifieke rol, zodat toegang en verantwoordelijkheden per project kunnen verschillen. De tabel `dependencies` bevat vervolgens de concrete dependencies per project, samen met versies, statusinformatie, AI samenvattingen en metadata zoals de changelog-URL en het tijdstip van de laatste controle. Ten slotte laat `invitations` toe om nieuwe gebruikers uit te nodigen voor een bedrijf via een tokengebaseerde uitnodiging. Figuur 5.2 toont hoe deze entiteiten samen het datamodel vormen waarop het dashboard steunt.



Figuur 5.2: Conceptueel ERD met bedrijven, projecten, leden en dependencies

De AI-laag bestaat uit drie parallelle paden die in aparte secties verder worden uitgewerkt. De Mastra-agent draait als TypeScript-service naast de Next.js-app en wordt aangeroepen via een interne API-route. De n8n-workflow draait in een eigen runtime en wordt aangesproken via een webhook-URL. De OpenAI-agent-builder wordt gebruikt om een gehoste agent te configureren met vergelijkbare tools en prompt. In alle gevallen verloopt de dataflow volgens hetzelfde patroon: dependencygegevens en eventueel changelog- en webcontext worden doorgestuurd naar de gekozen AI-component, de gestructureerde samenvatting komt terug naar de backend en wordt opgeslagen in Supabase. Hierdoor kan het dashboard de resultaten van de verschillende AI-aanpakken op een uniforme manier presenteren en vergelijken.

5.2. Implementatie van de Mastra-agent

Voor de eerste AI-integratie wordt gebruikgemaakt van Mastra, een TypeScript framework om AI-agents, tools en workflows te definiëren en te orchestreren. In de proof-of-concept wordt een `dependencyAgent` opgezet die specifiek is afgestemd op het analyseren van npm-dependency-updates. Deze agent gebruikt het taalmodel (OpenAI's `gpt-4o-mini`) met twee eigen tools: `fetchNpmInfoTool` en `web-`

`ChangelogSearchTool`.

De agent volgt een gestructureerde research-strategie: eerst haalt de `fetchNpm-InfoTool` metadata en releasenotes op uit npm en GitHub; als die incompleet zijn, volgt de `webChangelogSearchTool` met een gerichte webzoekopdracht naar changelogs. Het resultaat wordt gecombineerd en volgens een strikt JSON-formaat geformatteerd, zodat ontwikkelaars direct actiegerichte inzichten krijgen over features, breaking changes, security fixes en migratiestappen. Codefragment 5.1 toont de definitie van de agent en de system prompt.

```

1  import { Agent } from "@mastra/core/agent";
2  import { openai } from "@ai-sdk/openai";
3
4  export const dependencyAgent = new Agent({
5    id: "DependencyAnalyzer",
6    name: "Dependency Analyzer",
7    instructions: `
8    You are a dependency update analyst for software developers.
9    Your goal is to give developers CONCRETE, SPECIFIC information about
10     package updates
11     so they do NOT have to read the full changelog themselves.
12
13     ## Research Strategy (follow IN ORDER)
14     ### Step 1 – fetch-npm-info
15     Always start by calling the fetch-npm-info tool ...
16
17     ### Step 2 – web-changelog-search
18     Call when npm returns "No release notes found" ...
19
20     ## Output Format
21     ## [package-name] v[from] → v[to] ([major/minor/patch] update)
22     > Source: [tool source or URL]
23
24     ### New Features
25     ### Breaking Changes
26     ### Security Fixes
27     ### Deprecated
28     ### Migration Steps
29     ### Safe to update?
30     `,
31    model: openai("gpt-4o-mini"),
32    tools: {
33      fetchNpmInfoTool,
34      webChangelogSearchTool,
35    },
36  });

```

Codefragment 5.1: Configuratie van de dependencyAgent met de nodige tools voor de gekozen research strategie

De `fetchNpmInfoTool` (zie Codefragment 5.2) is de primaire tool voor het ophalen van npm-metadata en GitHub-release notes. De tool zoekt eerst naar een exacte release-tag (bijvoorbeeld `v1.2.3`), valt terug op de laatste vijf releases als die niet bestaat, en probeert vervolgens CHANGELOG bestanden (`CHANGELOG.md`, `HISTORY.md`, ...) uit de repository te lezen. Dankzij fallback logica en robuuste foutafhandeling levert de tool altijd een gestructureerd resultaat, zelfs als er geen officiële release

notes zijn.

```
export const fetchNpmInfoTool = createTool({
  id: "fetch-npm-info",
  inputSchema: z.object({
    packageName: z.string(),
    fromVersion: z.string(),
    toVersion: z.string(),
  }),
  execute: async ({ packageName, fromVersion, toVersion }) => {
    const npmData = await
      fetch(`https://registry.npmjs.org/${packageName}`).then(r =>
        r.json());
    const repoMatch = npmData.repository?.url?.match(/github\.com[/:]([\w.-]+)\/[\w.-]+\/);

    // exact tag, recente releases of CHANGELOG fallback
    if (repoMatch) {
      // ...
    }

    return {
      description: npmData.description || "",
      repository: npmData.repository?.url || "",
      releaseNotes: releaseNotes || "No release notes found on GitHub.",
      source,
    };
  },
});
```

Codefragment 5.2: Codefragment: fetchNpmInfoTool met npm-metadata en fallbacklogica

Als de npm-tool geen nuttige release notes vindt, volgt de webChangeLogSearchTool (zie Codefragment 5.3). Deze tool voert een gerichte webzoekopdracht uit naar changelogs via Tavily, Brave Search of DuckDuckGo, met domeinpriorisering voor officiële package-sites (bijvoorbeeld `next.js.org`, `angular.dev`). De tool prioriteert resultaten van officiële documentatie boven aggregators en levert een lijst van relevante titels, URL's en snippets.

```

export const webChangelogSearchTool = createTool({
  id: "web-changelog-search",
  inputSchema: z.object({
    packageName: z.string(),
    fromVersion: z.string(),
    toVersion: z.string(),
  }),
  execute: async ({ packageName, fromVersion, toVersion }) => {
    const query = buildQuery(packageName, fromVersion, toVersion);
    try {
      const provider = process.env.CHANGELOG_SEARCH_PROVIDER || "tavily"

      if (provider === "brave" && process.env.BRAVE_SEARCH_API_KEY) {
        return await searchWithBrave(query);
      }

      if (process.env.TAVILY_API_KEY) {
        return await searchWithTavily(query, packageName, toVersion);
      }

      return await searchWithDuckDuckGo(query);
    } catch (err) {
      return { results: [], query };
    }
  },
});

```

Codefragment 5.3: Codefragment: `webChangelogSearchTool` voor webzoekopdrachten naar changelogs

De Mastra-agent wordt vanuit de Next.js-backend aangeroepen via een API-route die de dependency-gegevens doorstuurt en de gestructureerde output opslaat in Supabase. Door de combinatie van deze twee tools en de expliciete instructies levert de agent betrouwbare, actionable samenvattingen die ontwikkelaars tijd besparen bij het beoordelen van updates.

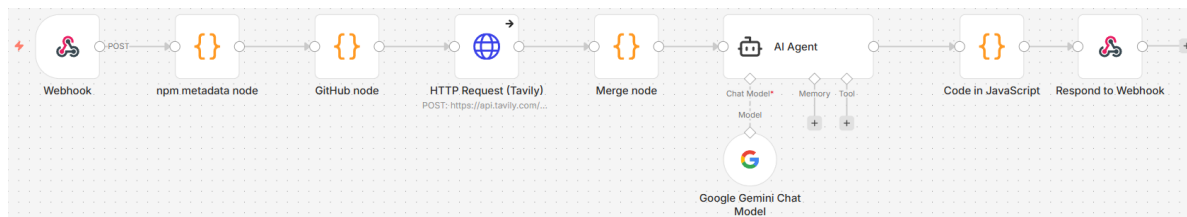
5.3. Implementatie van de n8n-workflow

Als tweede AI-integratie wordt een n8n-workflow opgezet die dezelfde kernfunctie vervult als de Mastra-agent, maar dan in een low-code, visueel georkestreerde omgeving. De workflow ontvangt een HTTP-aanvraag vanuit de webapplicatie, verkrijgt de aangeleverde dependency-informatie met data uit npm, Github en een webzoekdienst (Tavily)⁸, en laat vervolgens een AI-model (OpenAI's gpt-4o-mini)

⁸Tavily is een API en infrastructuurlaag die AI-agents en LLM-toepassingen real-time toegang geeft tot webzoekopdrachten, webextractie en crawling.<https://www.tavily.com/>

een samenvatting genereren van de relevante wijzigingen. Het resultaat wordt teruggestuurd naar de applicatie via de webhook-respons en opgeslagen in de databank.

De workflow bestaat uit een keten van acht nodes (zie Figuur 5.3):



Figuur 5.3: Overzicht van de n8n-workflow die metadata, release notes, webcontext en AI-samenvatting combineert

- **Node 1: Webhook.** Deze node fungeert als ingangspunt voor de workflow en wordt getriggerd zodra de Next.js-backend een HTTP-aanvraag naar de n8n-URL stuurt. De node is geconfigureerd om te antwoorden via een aparte *Respond to Webhook*-node, zodat alle tussenliggende stappen eerst afgewerkt worden.
- **Node 2: npm-metadata (Code).** In deze code-node wordt de JSON-body van de inkomende request uitgelezen (`packageName`, `fromVersion`, `toVersion`) en wordt via `this.helpers.httpRequest` een call gedaan naar `https://registry.npmjs.org/{packageName}`. De node reconstrueert de repository-URL uit de npm-metadata en geeft een nieuwe JSON-object door met omschrijving, homepage en repository-informatie.
- **Node 3: Github-node (Code).** Deze node ontvangt de verrijkte npm-data, extraheert `owner/repo` uit de repository-URL en probeert via de Github API eerst release notes op te halen voor de specifieke tag (`v{toVersion}` of `toVersion`). Als die niet gevonden wordt, geeft de node de laatste releases op en concateneert de body-teksten tot een `releaseNotes`-veld. Als er ook geen releases zijn, wordt er nog geprobeerd een `CHANGELOG.md`, `CHANGES.md` of `HISTORY.md` rechtstreeks uit de repository te lezen. De node voegt `releaseNotes`, `changeLogUrl` en `source` toe aan de JSON.
- **Node 4: HTTP Request (Tavily).** Via een HTTP-request-node wordt de Tavily API aangeroepen om webresultaten op te halen die extra context kunnen geven over de dependency of de update (bijvoorbeeld blogposts of documentatie rond de nieuwe versie).
- **Node 5: Merge-node (Code).** Deze node voegt de Github-data en Tavily-resultaten samen. De webresultaten worden omgezet naar een tekstveld `web-Snippets` (titel, URL en inhoud per resultaat), dat samen met de eerder verzamelde metadata en release notes wordt doorgegeven aan de AI-node.

- **Node 6: AI-agent (OpenAI).** Een AI-node, geconfigureerd met het model gpt-4o-mini, ontvangt releaseNotes, webSnippets en versie-informatie als input. Met behulp van een system prompt (zie Codefragment 5.4) en user prompt genereert het model een gestructureerde samenvatting van de update (bijvoorbeeld features, breaking changes, security-aspecten en migratieadvies).
- **Node 7: Code-node (post-processing).** Deze node extraheert de relevante output uit de AI-respons (bijvoorbeeld output of summary) en zet die om naar een eenvoudige JSON-object met één veld summary, zodat de webhook-respons helder en compact blijft.
- **Node 8: Respond to Webhook.** De laatste node stuurt de summary terug als HTTP-respons naar de oorspronkelijke aanroeper. Aan de kant van de webapplicatie kan deze samenvatting vervolgens rechtstreeks worden opgeslagen in de databank.

```

1 You are a dependency update analyst. Given structured data about a package
2 update, produce a concrete, developer-focused summary.
3
4 Format your response exactly like this:
5
6 ## [package] v[from] → v[to] ([major/minor/patch] update)
7
8 > Source: [source field from input]
9
10 #### New Features
11 #### Breaking Changes
12 #### Security Fixes
13 #### Deprecated
14 #### Migration Steps
15 #### Safe to update?
16
17 Base everything on the releaseNotes and webSnippets provided.
18 Never invent changes. If a section is empty, write 'None in this release.'
```

Codefragment 5.4: System prompt van de n8n Node 6 (Gemini AI-agent) met strikt outputformaat

De prompt zorgt ervoor dat het model een uniforme, actionable samenvatting genereert die direct bruikbaar is in het dashboard.

De Next.js-backend roept de n8n-workflow aan via een dedicated route (vereenvoudigd weergegeven in Codefragment 5.5). Deze route valideert eerst de gebruiker met Supabase, leest de dependencygegevens uit de inkomende request (dependencyId, name, currentVersion, latestVersion) en stuurt die als JSON-payload door naar

de n8n-webhook. De samenvatting die n8n terugstuurt wordt daarna opgeslagen in het veld `ai_summary` van de desbetreffende dependency.

```
1 export async function POST(request: Request) {
2   const supabase = await createClient();
3   const { data: { user } } = await supabase.auth.getUser();
4   if (!user) return NextResponse.json({ error: "Unauthorized" }, { status:
      401 });
5
6   const { dependencyId, name, currentVersion, latestVersion } = await
      request.json();
7
8   const n8nRes = await fetch(process.env.N8N_WEBHOOK_URL!, {
9     method: "POST",
10    headers: { "Content-Type": "application/json" },
11    body: JSON.stringify({ packageName: name, fromVersion: currentVersion,
      toVersion: latestVersion }),
12  });
13
14  const { summary } = await n8nRes.json();
15  await supabase.from("dependencies").update({ ai_summary: summary
      }).eq("id", dependencyId);
16
17  return NextResponse.json({ summary });
18 }
```

Codefragment 5.5: Codefragment: Next.js-route die de n8n-workflow aanroept

5.4. Implementatie van de OpenAI-agent-builder

Als derde AI-integratie wordt een eigen OpenAI-agent geïmplementeerd met de OpenAI API, die dependency updates analyseert via een expliciete tool loop. De agent gebruikt hetzelfde model als Mastra (gpt-4o-mini), gecombineerd met de tools `fetchNpmInfoTool` en `webChangeLogSearchTool`, en volgt een gelijkaardige research-strategie: eerst npm/GitHub metadata, dan webzoekopdrachten bij onvolledige info. Codefragment 5.6 toont de kernconfiguratie met system prompt en agent-loop.

```

1  const SYSTEM_PROMPT = `
2  You are a dependency update analyst for software developers.
3  ## Research Strategy (follow IN ORDER)
4  ### Step 1 – fetch-npm-info
5  Always start by calling the fetch-npm-info tool ...
6  ### Step 2 – web-changelog-search
7  Call when npm returns "No release notes found" ...
8  ## Output Format
9  ## [package] v[from] → v[to] ([major/minor/patch])
10 > Source: [URL]
11 ### New Features
12 ### Breaking Changes
13 ### Security Fixes
14 ### Deprecated
15 ### Migration Steps
16 ### Safe to update?
17 `;
18
19 export async function analyzeDependency({name, currentVersion,
20   latestVersion}) {
21   const messages = [{role: "system", content: SYSTEM_PROMPT}, {role:
22     "user", ...}];
23   // Agentic loop - max 10 turns
24   for (let turn = 0; turn < 10; turn++) {
25     const response = await client.chat.completions.create({
26       model: "gpt-4o-mini", messages, tools, tool_choice: "auto"
27     });
28     // Handle tool calls: execute & feed back results
29     if (choice.finish_reason === "stop") return assistantMessage.content;
30   }
31 }

```

Codefragment 5.6: OpenAI dependencyAgent met system prompt en tool-loop

De `fetchNpmInfoTool` (zie Codefragment 5.7) haalt npm-metadata op, zoekt GitHub releases (exact tag of recentste) en valt terug op CHANGELOG parsing met fallback logica voor robuustheid.

```

export async function executeFetchNpmInfo(args) {
  const npmData = await
    fetch(`https://registry.npmjs.org/${args.packageName}`).then(r =>
      r.json());
  const repoMatch =
    npmData.repository?.url?.match(/github\.com[/:]([\w.-]+\w[\w.-]+)/);
  if (repoMatch) {
    // Exact tag, recent releases, CHANGELOG.md fallback
    for (const tag of [`v${args.toVersion}`, args.toVersion]) {
      const res = await fetch(`https://api.github.com/repos/${repoMatch[1]},
        /releases/tags/${tag}`);
      if (res.ok && data.body) { /* use release notes */ }
    }
  }
  return { description: npmData.description || "", releaseNotes:
    releaseNotes || "No release notes found" };
}

```

Codefragment 5.7: fetchNpmInfoTool: npm-metadata en GitHub-releases met CHANGELOG-fallback

Als npm onvoldoende releasenotes vindt, activeert de webChangelogSearchTool (zie Codefragment 5.8) een webzoekopdracht via Tavily, Brave of DuckDuckGo, met domeinprioriteit voor officiële sites zoals `nextjs.org` of `angular.dev`

```

export async function executeWebChangelogSearch(args) {
  const query = buildQuery(args.packageName, args.fromVersion,
    args.toVersion);
  try {
    const provider = process.env.CHANGELOG_SEARCH_PROVIDER || "tavily";

    if (provider === "brave" && process.env.BRAVE_SEARCH_API_KEY) {
      return await searchWithBrave(query);
    }
    if (process.env.TAVILY_API_KEY) {
      return await searchWithTavily(query, packageName);
    }
    return await searchWithDuckDuckGo(query);
  } catch {
    return { results: [], query };
  }
}

```

Codefragment 5.8: webChangelogSearchTool: webzoekopdrachten naar changelogs

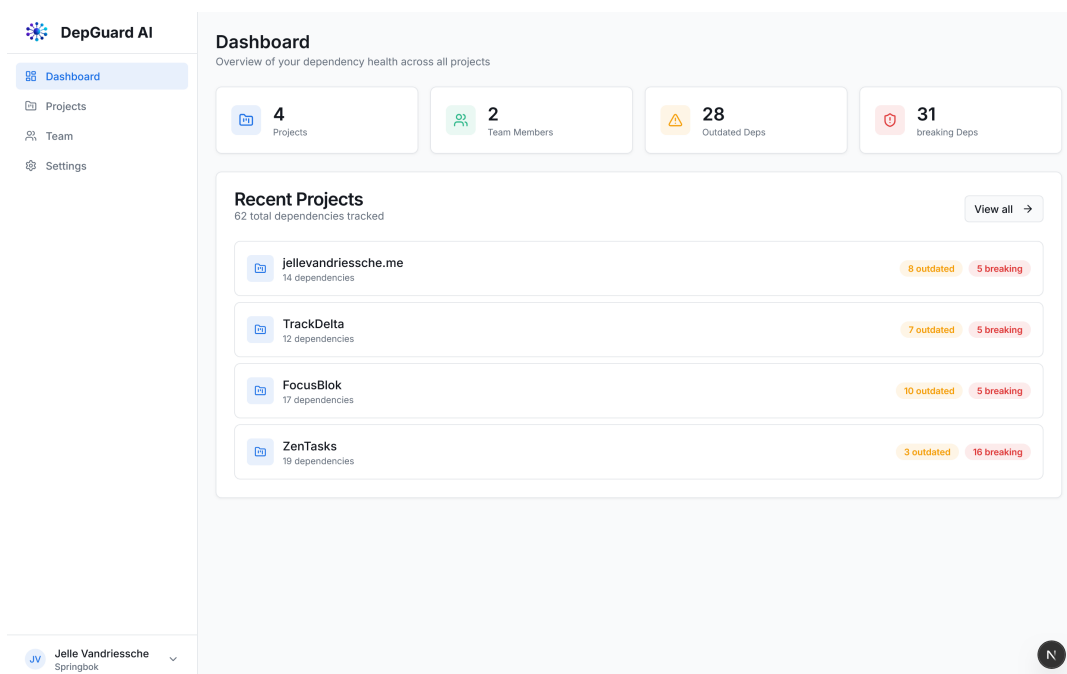
De Next.js backend roept de OpenAI agent aan via een API route, analoog aan

Mastra en n8n: dependencygegevens worden doorgestuurd, de gestructureerde samenvatting opgeslagen in Supabase, en via het dashboard gepresenteerd. Deze custom implementatie biedt maximale controle over de toolflow en outputformaat, ideaal voor vergelijking met de framework-gebaseerde Mastra en n8n aanpakken.

5.5. Integratie met de webapplicatie

Deze sectie beschrijft hoe de verschillende onderdelen van DepGuard AI samenkomen in de webapplicatie en hoe dependencygegevens, AI-analyses en gebruikersinteracties in een uniforme gebruikerservaring worden geïntegreerd. De integratie is zodanig opgezet dat de frontend niet rechtstreeks communiceert met externe diensten of AI-componenten, maar steeds via de backendlaag werkt. Dat maakt het mogelijk om authenticatie, validatie, foutafhandeling en opslag centraal te beheren.

De webapplicatie vormt het centrale toegangspunt voor eindgebruikers. Vanuit het dashboard kunnen gebruikers projecten beheren, repositories koppelen, dependencyscans opstarten en de resultaten van AI-analyses bekijken. Daarbij fungeert de frontend in hoofdzaak als presentatielaag: gebruikersacties worden doorgestuurd naar de backendroutes, waarna de backend de nodige gegevens ophaalt, verwerkt en opnieuw in een gestandaardiseerd formaat aan de interface aanbiedt. Deze scheiding tussen gebruikersinterface en serverlogica verhoogt de onderhoudbaarheid en maakt het mogelijk om verschillende AI-implementaties op een uniforme manier in dezelfde applicatie te integreren.



Figuur 5.4: Dashboardoverzicht van DepGuard AI met projectbeheer en statusindicatoren per repository.

Wanneer een gebruiker een project opent, haalt de applicatie eerst de bijhorende project- en dependencygegevens op uit Supabase. Per dependency worden onder meer de pakketnaam, huidige versie, laatst gekende versie, status en eventuele AI-samenvatting weergegeven. De frontend groepeerde deze informatie in een dashboardoverzicht, zodat gebruikers snel kunnen zien welke dependencies verouderd zijn en welke updates bijkomende aandacht vereisen. Indien voor een dependency al een analyse beschikbaar is, wordt die rechtstreeks vanuit de databank geladen en in de interface getoond, zonder dat de AI-component opnieuw moet worden aangeroepen.

The screenshot shows the DepGuard AI interface for a project named 'TrackDelta'. The left sidebar contains navigation links: Dashboard, Projects (selected), Team, and Settings. The main content area displays the project name 'TrackDelta' with a description: 'A real-time Formula 1 telemetry dashboard built with FastF1, Next.js, and Tailwind CSS. Explore lap times, sector deltas, and driver performance through interactive data visualizations.' Below this, there are buttons for 'Upload', 'Re-scan', and 'Delete'. A summary bar shows 'All (12)', 'Current (0)', 'Outdated (7)', and 'Breaking (5)' dependencies. The 'Dependencies (12)' section includes a search bar and a table listing dependencies with their current and latest versions, update status, and AI analysis options.

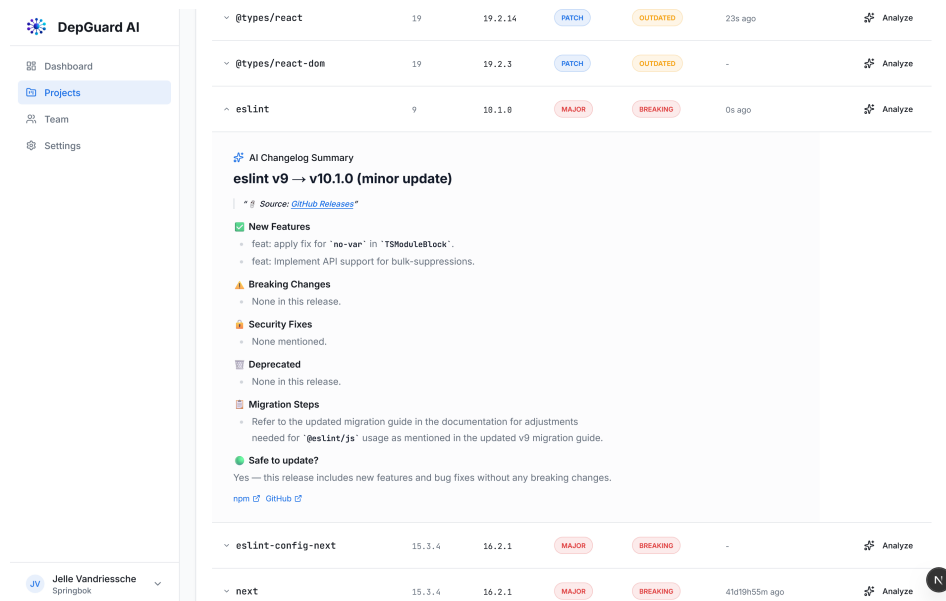
Package	Current	Latest	Update	Status	Last AI	Actions
✓ @eslint/eslintre	3	3.3.5	PATCH	OUTDATED	-	Analyze
✓ @tailwindcss/postcss	4	4.2.2	PATCH	OUTDATED	-	Analyze
✓ @types/node	20	25.5.0	MAJOR	BREAKING	-	Analyze
✓ @types/react	19	19.2.14	PATCH	OUTDATED	-	Analyze
✓ @types/react-dom	19	19.2.3	PATCH	OUTDATED	-	Analyze
✓ eslint	9	10.1.0	MAJOR	BREAKING	-	Analyze
✓ eslint-config-next	15.3.4	16.2.1	MAJOR	BREAKING	-	Analyze
✓ next	15.3.4	16.2.1	MAJOR	BREAKING	41d19h53m ago	Analyze

Figuur 5.5: Detailpagina van een project met de dependencylijst, versiestatus en beschikbare AI-analyses.

Het opstarten van een nieuwe analyse verloopt op aanvraag vanuit de gebruikers-interface. Wanneer een gebruiker een dependency of project laat analyseren, verstuurt de frontend een verzoek naar een backendroute met de relevante parameters, zoals de pakketnaam en de huidige en nieuwste versie. De backend bepaalt vervolgens welke AI-variant beschikbaar is, verrijkt indien nodig de input met bijkomende metadata en stuurt de aanvraag door naar de juiste AI-component. Na verwerking wordt de teruggestuurde samenvatting opgeslagen in de tabel dependencies, zodat de analyse later opnieuw kan worden geraadpleegd zonder de volledige verwerkingsketen te herhalen.

De resultaten van de AI-analyse worden in de webapplicatie niet enkel als vrije tekst weergegeven, maar ingebed in de bredere context van dependencybeheer. De samenvatting wordt gekoppeld aan de desbetreffende dependency en verschijnt samen met versie-informatie, updateclassificatie en eventuele bronverwijzingen,

zoals een changelog-URL. Daardoor krijgt de gebruiker niet alleen een tekstuele uitleg van de wijziging, maar ook een concreet overzicht van welke dependency het betreft en welke bijkomende context beschikbaar is. Die koppeling is essentieel om AI-output bruikbaar te maken binnen een dashboardomgeving, aangezien de interpretatie pas waardevol wordt wanneer ze verbonden is met de operationele gegevens van het project.



Figuur 5.6: AI-samenvatting van een dependency-update, weergegeven samen met versie-informatie, updateclassificatie en changelog-verwijzing.

Naast het ophalen en tonen van analyses ondersteunt de integratie ook de consistente vergelijking tussen de verschillende AI-aanpakken. Omdat alle varianten uiteindelijk een samenvatting in een gelijkaardig formaat teruggeven en die via dezelfde backendlaag in Supabase worden opgeslagen, kan het dashboard deze resultaten op een uniforme manier presenteren. Dat maakt het mogelijk om de verschillende implementaties binnen een applicatiecontext te evalueren, zonder voor elke aanpak een afzonderlijke gebruikersinterface te moeten uitwerken.

De integratie met de webapplicatie vervult daarmee een dubbele rol. Enerzijds zorgt ze ervoor dat de gebruikers op een toegankelijke manier met dependency-gegevens en AI-analyses kunnen werken. Anderzijds vormt ze de technische verbindingslaag die de verschillende backend- en AI-componenten samenbrengt tot een samenhangend systeem. Hierdoor wordt DepGuard AI niet louter een verzameling losse analysecomponenten, maar een geïntegreerd dashboard dat projectgegevens, update-informatie en AI-ondersteuning samen aanbiedt in een workflow.

6

Resultaten en evaluatie

Dit hoofdstuk bespreekt de resultaten van de uitgevoerde evaluatie van de drie AI-varianten binnen DepGuard AI. De analyse vertrekt vanuit de benchmarkuitvoer van de proof-of-concept en vergelijkt de oplossingen op het vlak van specificiteit, volledigheid, actiegerichtheid en verwerkingstijd. Op basis daarvan wordt nagegaan in welke mate de verschillende benaderingen bruikbare ondersteuning bieden voor dependencybeheer in de context van Springbok.

6.1. Evaluatiekader

Vooraleer de resultaten worden besproken, wordt in deze sectie toegelicht hoe de evaluatie is opgezet. Eerst wordt de gebruikte scoremethode *LLM-as-a-Judge* beschreven, inclusief de beoordelingsdimensies, gewichten en scoringsformules. Daarna worden de vijf testcases geïntroduceerd en gemotiveerd, met per testcase een overzicht van de verwachte output op basis van de officiële changelog.

6.1.1. Scoremethode: LLM-as-a-Judge

Om de output van de drie AI-agents op een consistente en herhaalbare manier te beoordelen, maakt de benchmark gebruik van de *LLM-as-a-Judge*-aanpak. Bij deze evaluatiemethode treedt een tweede taalmodel op als rechter en beoordeelt de output van de agent semantisch, zonder te vertrekken van vaste regelsets of eenvoudige keyword-matching. De judge begrijpt de *betekenis* van de output en vergelijkt die inhoudelijk met de verwachte kwaliteit en feiten. Dit is precies wat LLM-as-a-Judge onderscheidt van klassieke evaluatiemethodes.

In deze benchmark is het judge-model *gpt-4o-mini* geconfigureerd met een lage temperatuur (*temperature* = 0,1)¹ om consistente en herhaalbare oordelen te ga-

¹De temperatuurparameter bepaalt hoeveel variatie een taalmodel genereert: een lagere waarde zorgt voor minder willekeurige en beter reproduceerbare output.

randeren. Elke agent-output wordt beoordeeld op vier kwantitatieve dimensies (schaal 0–10) en twee kwalitatieve indicatoren:

1. **Specificiteit** (30 % gewicht): meet of de analyse concrete, package-specifieke informatie bevat. Lage scores wijzen op vage of generieke tekst; hoge scores impliceren exacte API-namen, versienummers of codefragmenten.
2. **Actiegerichtheid** (40 % gewicht): meet of een developer op basis van deze output een gefundeerde beslissing kan nemen over de update, zonder zelf de changelog te raadplegen. Dit is de zwaarst wegende dimensie, omdat het de kernvraag van het systeem beantwoordt.
3. **Volledigheid** (15 % gewicht): meet of alle verwachte secties aanwezig zijn: nieuwe functies, breaking changes, migratiestappen en een expliciete veiligheidsinschatting.
4. **Feitendekking** (15 % gewicht): meet hoeveel van de vooraf gedefinieerde verwachte feiten (*expected facts*) de agent correct en volledig vermeldt. Hiermee wordt de output getoetst aan de grondwaarheid uit de officiële changelog.

De *LLM-as-a-Judge* kent deze scores toe op basis van een vaste beoordelingsprompt waarin per testcase de verwachte feiten, de vereiste outputstructuur en de beoordelingscriteria expliciet worden meegegeven. De judge vergelijkt de gegenereerde output dus niet alleen semantisch met de officiële changelog, maar beoordeelt ook of de response voldoende specifiek, volledig en actiegericht is volgens een vooraf vastgelegde rubric. De score is bijgevolg een modelgestuurde interpretatie van inhoudelijke overeenkomst en bruikbaarheid, en geen eenvoudige trefwoordtelling. Het model werd dus niet apart getraind op deze benchmark, maar beoordeelde elke output op basis van de meegegeven instructies, verwachte feiten en scoring-rubric.

De totale score per run wordt berekend als gewogen gemiddelde:

$$\begin{aligned}
 \text{overall} &= \text{actiegerichtheid} \times 0,40 \\
 &\quad + \text{specificiteit} \times 0,30 \\
 &\quad + \text{volledigheid} \times 0,15 \\
 &\quad + \text{feitendekking} \times 0,15
 \end{aligned}
 \tag{6.1}$$

Naast de vier kwantitatieve dimensies registreert de judge ook twee booleaanse indicatoren en een categorisch veiligheidsoordeel. De booleaanse indicatoren geven aan of de output een expliciete sectie over *breaking changes* bevat en of concrete *migration steps* aanwezig zijn. Daarmee wordt nagegaan of de agent niet alleen inhoudelijk relevante informatie vermeldt, maar die ook in een bruikbare, gestructureerde vorm presenteert. Het veiligheidsoordeel geeft vervolgens aan hoe de

agent de update evalueert: *Yes* betekent dat de update veilig lijkt, *No* dat ze afgeraden wordt, *With caution* dat voorzichtigheid vereist is, en *Not found* dat geen expliciet oordeel werd gegeven. Elk testgeval wordt driemaal uitgevoerd per agent, waarna de gerapporteerde scores de rekenkundige gemiddelden zijn over die drie runs. De consistentiescore meet de stabiliteit van een agent over deze herhalingen:

$$\text{consistentie} = \max(0, 10 - \sigma_{\text{overall}} \times 2) \quad (6.2)$$

waarbij σ_{overall} de standaardafwijking is van de overallscores. Een agent die sterk varieert tussen runs scoort lager op consistentie en is in productie minder betrouwbaar.

6.1.2. Testcaseselectie en verwachte output

De benchmark omvat vijf testcases die samen de volledige breedte van updatetypes *patch*, *minor* en *major* dekken, verspreid over uiteenlopende packagecategorieën en risiconiveaus. De selectie is als volgt gemotiveerd:

- **react minor (18.2.0 → 18.3.0):** React is een van de meest gebruikte frontend bibliotheken en een frequent terugkerende dependency in projecten zoals die van Springbok. Deze minor update voegt functioneel niets toe maar introduceert uitsluitend deprecatiewaarschuwingen voor React 19. Het is daarmee een *edge case*: agents die de update simpelweg als “veilig zonder bijzonderheden” beschrijven missen essentiële informatie voor toekomstgericht onderhoud.
- **next.js minor (14.1.0 → 14.2.0):** Next.js is een veelgebruikte meta-framework in productieomgevingen. Deze minor update brengt aanzienlijke prestatie- en stabiliteitsverbeteringen met zich mee (Turbopack, geheugenverbruik). Agents worden getest op hun vermogen om niet-triviale verborgen meerwaarde te herkennen en te rapporteren.
- **TypeScript major (4.9.5 → 5.0.4):** Een major update met reële breaking changes (nieuwe decoratorstandaard, deprecated tsconfig-opties, gewijzigde minimumversies). Dit is de moeilijkste testcase en meet de kwaliteit van risicoanalyse bij updates met hoge impact.
- **zod patch (3.22.3 → 3.22.4):** Een patch binnen een stabiele minor-lijn waarvoor de officiële changelog geen gedetailleerde patchnotes publiceert. Dit dient als *eerlijkheidstest*: een betrouwbare agent vermeldt dat specifieke details niet beschikbaar zijn, in plaats van concrete bugfixes te verzinnen.
- **tailwindcss minor (3.3.0 → 3.4.0):** Een minor update met uitgebreide nieuwe utilities (dynamische viewport-eenheden, *has-**-variant, *size-**-utilities, sub-grid) die volledig backwards compatibel zijn. Agents worden getest op hun

vermogen om een brede set nieuwe functies correct en volledig te rapporteren.

De vijf testcases bestrijken samen drie risiconiveaus (patch, minor, major), vijf packagedomeinen (UI-framework, meta-framework, typesysteem, validatiebibliotheek, CSS-framework) en twee bijzondere scenario's (deprecation warnings zonder breaking changes; patch zonder publieke notes). Tabel 6.1 vat de details samen.

ID	Package	Versieovergang	Type	Verwacht oordeel & kernpunten
minor	react	18.2.0 → 18.3.0	minor	Yes — alleen deprecatiewaarschuwingen; geen nieuwe functies
minor	nextjs	14.1.0 → 14.2.0	minor	Yes — Turbopack-verbeteringen, drastische geheugenreductie
major	typescript	4.9.5 → 5.0.4	major	<i>With caution</i> — nieuwe decorators, deprecated tsconfig-opties
patch	zod	3.22.3 → 3.22.4	patch	Yes — geen publieke patchnotes; eerlijkheidstest
minor	tailwindcss	3.3.0 → 3.4.0	minor	Yes — uitgebreide nieuwe utilities, volledig compatibel

Tabel 6.1: Overzicht van de gebruikte testcases met verwacht veiligheidsoordeel

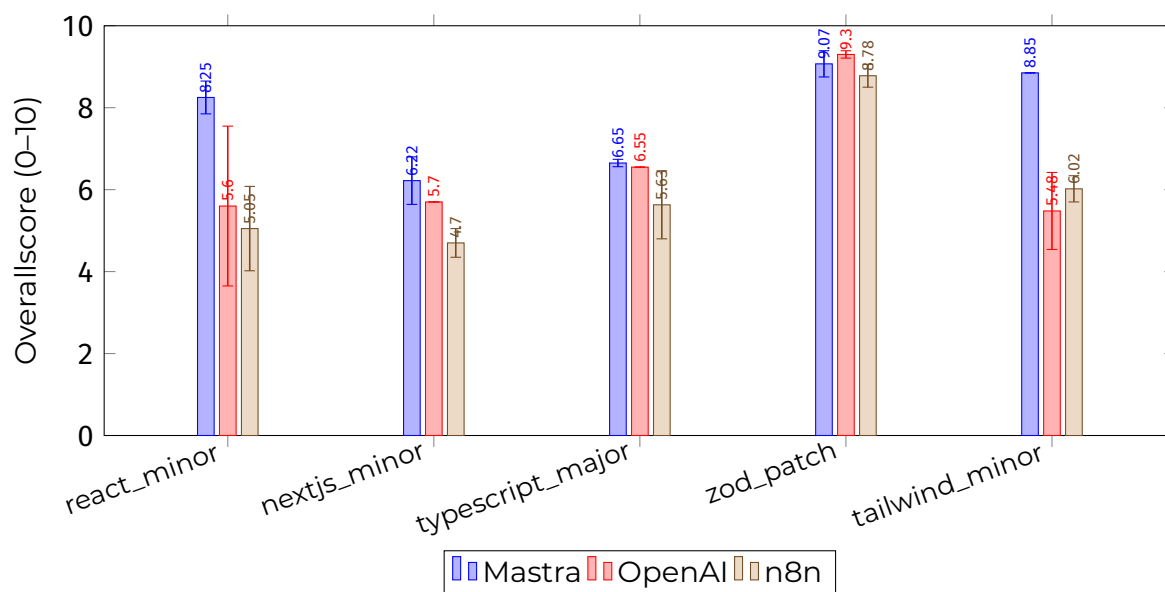
6.2. Globale resultaten

De benchmark werd uitgevoerd op 17 mei 2026 en omvat in totaal 45 evaluaties: 5 testcases × 3 agents × 3 runs. Alle gerapporteerde scores zijn rekenkundige gemiddelden over de drie runs. Tabel 6.2 vat de geaggregeerde resultaten samen per variant.

Variant	Overall	Specificiteit	Volledigheid	Actiegerichtheid	Gem. latency
Mastra	7,81 ± 1,24	7,27 ± 1,33	8,20 ± 0,56	8,27 ± 1,28	10,94 ± 3,08 s
OpenAI	6,53 ± 1,70	5,60 ± 1,64	8,00 ± 1,31	7,27 ± 1,71	4,34 ± 1,19 s
n8n	6,04 ± 1,59	4,67 ± 2,09	6,67 ± 1,23	7,47 ± 1,19	8,07 ± 2,72 s

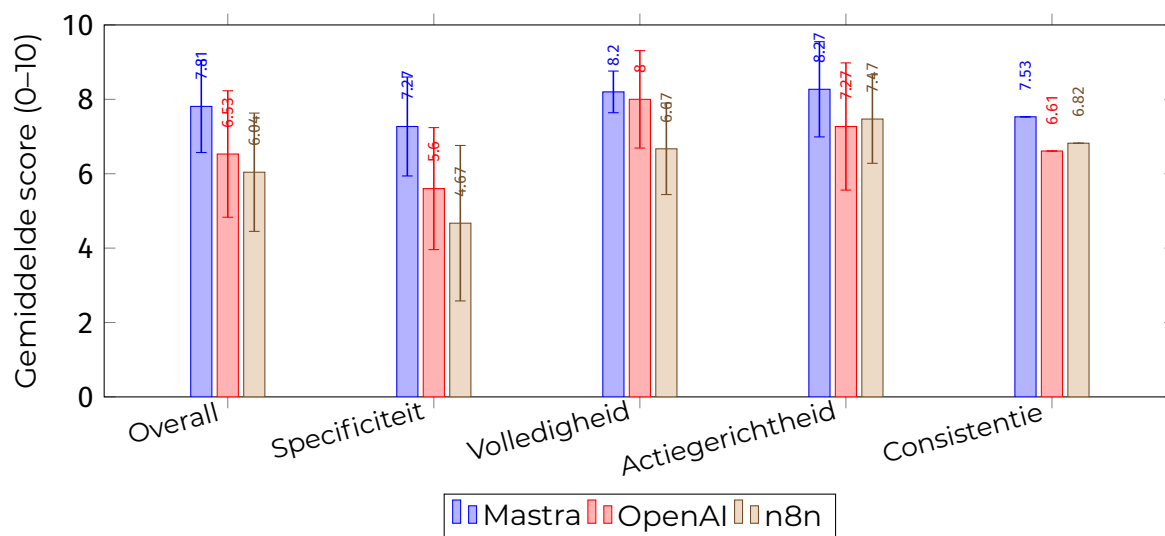
Tabel 6.2: Samenvatting van de benchmarkresultaten per AI-variant (gemiddelden ± σ over 5 testcases × 3 runs)

Figuur 6.1 toont de overallscores per testcase en per agent. Hieruit blijkt dat de drie varianten sterk uiteenlopen naargelang het type update, en dat geen enkele variant in alle testcases dominant is.



Figuur 6.1: Gemiddelde overallscores per testcase en per agent (gemiddeld over 3 runs; foutbalken = σ). De grafiek laat zien dat de prestaties sterk verschillen per update-type en dat geen enkele variant in alle testcases dominant is.

Figuur 6.2 vergelijkt de gemiddelde scores per beoordelingsdimensie over alle testcases.



Figuur 6.2: Gemiddelde scores per beoordelingsdimensie over alle testcases (foutbalken = σ ; geen σ beschikbaar voor consistentie). Hieruit blijkt dat Mastra het sterkst scoort op specificiteit, volledigheid en actiegerichtheid, terwijl n8n op alle dimensies lager uitkomt.

Mastra scoort het hoogst op overall (7,81), specificiteit (7,27), volledigheid (8,20) en actiegerichtheid (8,27). De agent haalt ook de hoogste consistentiescore (7,53), wat aangeeft dat de output over meerdere runs stabiel blijft. Het nadeel is de aanzienlijk hogere latency: gemiddeld 10,94 s tegenover 4,34 s voor OpenAI. OpenAI scoort beduidend lager op specificiteit (5,60) en actiegerichtheid (7,27), maar is de snelste

variant. n8n presteert op alle dimensies het laagst, met name op specificiteit (4,67) en volledigheid (6,67).

6.2.1. Statistische vergelijking van de agents

Om na te gaan of de waargenomen verschillen tussen de drie agents ook statistisch relevant zijn, werd per scorecategorie een one-way ANOVA uitgevoerd met *agent* als factor. Deze analyse werd apart toegepast op de overall score, specificiteit, volledigheid, actiegerichtheid en latency. De factor *testcase* werd in deze evaluatie niet expliciet gemodelleerd, waardoor de resultaten vooral als een eerste indicatie van groepsverschillen moeten worden geïnterpreteerd.

Voor de overall score werd een significant verschil gevonden tussen de agents ($F(2, 42) = 5,41, p = 0,0081$). Voor specificiteit werd eveneens een significant verschil gevonden ($F(2, 42) = 8,82, p = 0,00063$). Ook voor volledigheid was het verschil statistisch significant ($F(2, 42) = 8,80, p = 0,00064$). Voor actiegerichtheid werd geen statistisch significant verschil gevonden ($F(2, 42) = 2,11, p = 0,134$). Voor latency was het verschil duidelijk significant ($F(2, 42) = 26,97, p < 0,001$).

Omdat de benchmarkscore per testcase discreet is en de testcases inhoudelijk sterk verschillen, moet deze analyse voorzichtig worden geïnterpreteerd. Een two-way ANOVA of mixed model met *testcase* als bijkomende factor zou methodologisch strikter zijn, maar ligt buiten de scope van deze bachelorproef. De huidige analyse volstaat wel om de dominante trends in de benchmarkresultaten te onderbouwen.

6.3. Analyse per testcase

De vijf testcases worden hieronder afzonderlijk besproken. Per testcase wordt eerst de verwachte output beschreven op basis van de handmatig samengestelde *expected facts* uit de officiële changelog. Daarna worden de scores en de kwalitatieve bevindingen per agent toegelicht, met aandacht voor wat correct werd gerapporteerd, wat ontbrak en waar hallucinaties optraden.

6.3.1. React minor (18.2.0 → 18.3.0)

Verwachte output: de agent dient te vermelden dat deze release functioneel identiek is aan 18.2.0, uitsluitend deprecation warnings toevoegt, en dat de update veilig is.

Voorbeelden zijn onder meer `ReactDOM.render`, `ReactDOM.hydrate` en `defaultProps` op functiecomponenten.

Resultaten: Mastra behaalt een gemiddelde overallscore van 8,25 en scoort het best op deze testcase. De agent identificeert de deprecaties correct en vermeldt dat er geen breaking changes zijn. In twee van drie runs noemt Mastra echter `findDOMNode` en `test-utils` als deprecated, wat niet terugkomt in de officiële changelog – een voorbeeld van lichte hallucinatie. OpenAI scoort gemiddeld 5,60 en ver-

zint in twee van drie runs nieuwe functies die helemaal niet in React 18.3.0 zijn geïntroduceerd (o.a. `useTransition` en `startTransition`, die al in React 18.0 bestonden). n8n (gem. 5,05) geeft in de meeste runs een sjabloonmatige output waarbij alle secties als “None in this release” worden ingevuld, waardoor de relevante deprecatieinformatie volledig ontbreekt.

6.3.2. Next.js minor (14.1.0 → 14.2.0)

Verwachte output: de agent dient de Turbopack-stabilisering, de drastische geheugenreductie bij production builds (van ~2,2 GB naar <190 MB), de verbeterde hydration error messages en het nieuwe `staleTimes`-configuratieveld te vermelden. Er zijn geen breaking changes.

Resultaten: Dit is de moeilijkste minor-testcase, met de laagste scores voor de drie agents. Mastra behaalt een gemiddelde overallscore van 6,22, maar hallucineerde in twee runs breaking changes die niet in de changelog voorkomen (o.a. de verwijdering van de squoosh-afhankelijkheid). OpenAI scoort gemiddeld 5,70 en vermeldt eveneens incorrecte breaking changes. Beide agents missen de kernboodschap over de geheugenverbetering, die in productie potentieel de meeste waarde heeft. n8n (gem. 4,70) geeft de meest oppervlakkige output en mist migratiestappen in twee van drie runs.

6.3.3. TypeScript major (4.9.5 → 5.0.4)

Verwachte output: de agent dient de nieuwe ECMAScript Stage 3 decorators, de incompatibiliteit met legacy decorator-bibliotheken (`experimentalDecorators: true` blijft vereist voor NestJS/TypeORM), de deprecated `tsconfig`-opties, de gewijzigde minimumversies en het correct veiligheidsoordeel *With caution* te vermelden.

Resultaten: Alle drie de agents herkennen dat het om een risicovolle major update gaat. Mastra (gem. 6,65) en n8n (gem. 5,63) geven het juiste oordeel *With caution*; OpenAI geeft in alle drie de runs het oordeel *No*, wat te conservatief is. De nieuwe ECMAScript decorators *het meest impactvolle onderdeel van TypeScript 5.0* worden door Mastra wel vernoemd, maar ontbreken bij n8n en worden door OpenAI slechts gedeeltelijk beschreven. De specifieke impact op legacy-bibliotheken (NestJS, TypeORM, Angular) wordt door geen enkele agent voldoende concreet uitgewerkt.

6.3.4. Zod patch (3.22.3 → 3.22.4)

Verwachte output: de agent dient te vermelden dat er geen nieuwe functies of breaking changes zijn, en expliciet aan te geven dat de officiële changelog geen gedetailleerde patchnotes publiceert voor individuele patch-releases. Agents die toch concrete bugfixes opsommen zonder bron, fabriceren informatie.

Resultaten: Dit is de testcase waarop alle agents het best presteren: Mastra 9,07, OpenAI 9,30, n8n 8,78. Alle agents geven een duidelijk en correct Yes-oordeel. OpenAI scoort hier het hoogst dankzij een bijzonder volledige outputstructuur. De eer-

lijkheidstest wordt grotendeels geslaagd: in geen enkele run werden gefabriceerde details vastgesteld. Wel vermeldt OpenAI in twee runs “specifieke bugfixes” zonder bronverwijzing, wat de judge als lichte zwakheid beschouwt maar niet als expliciete hallucinatie.

6.3.5. Tailwind CSS minor (3.3.0 → 3.4.0)

Verwachte output: de agent dient de uitgebreide set nieuwe utilities te benoemen: dynamische viewport-eenheden (dvh, lvh, svh), de `has-*`-variant, `size-*`-utilities, `text-balance/text-pretty`, subgrid-ondersteuning en de uitgebreide doorzichtheidsschaal. Er zijn geen breaking changes.

Resultaten: Mastra domineert duidelijk met een gemiddelde overallscore van 8,85 – de hoogste score voor enige agent op enige testcase. Mastra haalt live changelog-data op via de `fetch-npm-info`-tool en slaagt erin de meeste verwachte utilities correct te benoemen. OpenAI (gem. 5,48) mist het overgrote deel van de nieuwe functies en noemt ten onrechte breaking changes. n8n (gem. 6,02) vermeldt in twee runs gefabriceerde details over een “devtools-integratie” die niet in de officiële changelog staat.

6.4. Analyse per agent

Mastra presteert het sterkst over alle testcases en dimensies. De drie agents beschikken over dezelfde tools een `npm`-informatietool en een zoektool maar Mastra verwerkt de opgehaalde tool-output structureel effectiever en integreert die beter in de uiteindelijke respons. Dit verklaart de grote voorsprong bij de `tailwind_minor`-testcase en de goede prestaties bij `react_minor`. De keerzijde is de hogere latency (gem. 10,94 s, p95 = 15,79 s), wat in een interactieve dashboardcontext merkbaar is. Mastra hallucineert ook sporadisch details die niet in de changelog staan, wat aangeeft dat de tool-output niet altijd volledig is en de agent soms ontbrekende informatie aanvult met niet-geverifieerde informatie.

OpenAI levert de snelste responses (gem. 4,34 s) en scoort uitstekend bij de `zod_patch` testcase. Tegelijk presteert de variant beduidend zwakker wanneer de update rijk is aan nieuwe functies of subtiele details, zoals bij `tailwind_minor` en `react_minor`. Hoewel de variant over dezelfde tools beschikt als Mastra, lijkt de orchestratie de opgehaalde informatie minder volledig te benutten, wat resulteert in meer hallucinaties en onvolledige secties. Het incorrecte *No*-oordeel bij de TypeScript major update (in plaats van *With caution*) is een kwalitatief significante fout: developers zouden op basis van dat oordeel de update mogelijk vermijden terwijl ze hem met aangepaste configuratie veilig kunnen doorvoeren.

n8n presteert het laagst op specificiteit (4,67) en volledigheid (6,67). De agent geeft in de meeste runs een structureel correcte output, maar de inhoud is schaars: bij de `react_minor`- en `nextjs_minor`-testcases worden alle secties als leeg gerapporteerd, ook al bevat de update relevante informatie. n8n haalt ook de laagste score

op het aanwezig zijn van migratiestappen (66,67 % tegenover 86,67 % voor Mastra en 100 % voor OpenAI). Anderzijds is de consistentiescore (6,82) iets hoger dan die van OpenAI (6,61), wat aangeeft dat de outputs van n8n minder variëren – al is dat deels te verklaren doordat de outputs structureel te oppervlakkig zijn.

6.5. Interpretatie van de resultaten

De benchmarkresultaten tonen aan dat de kwaliteit van AI-gestuurde dependencyanalyse sterk afhangt van de implementatievorm. Hoewel Mastra en OpenAI gebruikmaken van dezelfde ondersteunende tools, verschilt de manier waarop de opgehaalde informatie wordt georkestreerd en verwerkt. Mastra scoort vooral beter op specificiteit, volledigheid en actiegerichtheid, terwijl OpenAI sneller werkt en in sommige gevallen consistente output levert, maar minder diepgaande en minder accurate samenvattingen produceert bij updates met veel informatie.

De statistische vergelijking bevestigt dit algemene beeld: de verschillen tussen de agents zijn niet enkel descriptief, maar ook zichtbaar op meerdere scorecategorieën. Vooral de combinatie van inhoudelijke kwaliteit en verwerkingstijd laat zien dat de keuze voor een implementatie een duidelijke afweging inhoudt tussen nauwkeurigheid, volledigheid en snelheid.

Een opvallende bevinding is dat alle varianten het best presteren op de `zod_patch`-testcase, niet omdat die inhoudelijk het rijkst is, maar omdat het ontbreken van publieke patchnotes merkwaardig genoeg de hallucinatiedruk verlaagt: er is minder ruimte om foutieve details te introduceren. De moeilijkste testcase blijkt de `nextjs_minor`-update, waar alle agents incorrecte breaking changes rapporteren, vermoedelijk omdat de grens tussen een aanzienlijke verbetering en een potentieel brekende wijziging moeilijk correct te interpreteren is, zelfs wanneer de changelog beschikbaar is.

De resultaten ondersteunen de hypothese dat een AI-gestuurde analyselaag waardevol kan zijn voor dependencybeheer, maar benadrukken tegelijk dat de praktische meerwaarde niet uitsluitend bepaald wordt door het gebruikte taalmodel, maar ook door de beschikbaarheid van live changelog data, de kwaliteit van de outputstructuur en de robuustheid van de fallbacklogica bij ontbrekende releasenotes. Voor de context van Springbok betekent dit dat de keuze voor een implementatie een afweging vraagt tussen inhoudelijke kwaliteit, responssnelheid en onderhoudseenvoud.

7

Conclusie

Deze bachelorproef onderzocht in welke mate een AI-gedreven agent kan worden ingezet om dependency-updates te analyseren en te beheren, zodat het kernteam van Springbok dit proces efficiënter en met minder handmatig werk kan uitvoeren. De resultaten tonen aan dat een agentische aanpak een duidelijke meerwaarde biedt, omdat ze changelogs, release notes en versie-informatie kan samenbrengen tot een bruikbare samenvatting voor ontwikkelaars.

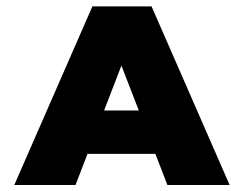
Uit het onderzoek blijkt dat het kernprobleem bij dependencybeheer niet zozeer de detectie van updates is, maar vooral de interpretatie ervan. Door automatische detectie te combineren met een AI-gestuurde analyse verlaagt DepGuard AI de drempel om updates te beoordelen en helpt het systeem ontwikkelaars sneller en beter geïnformeerde beslissingen te nemen. De implementatie en evaluatie tonen daarnaast aan dat de kwaliteit van de uitkomst sterk afhangt van de gekozen orkestratie en de manier waarop broninformatie wordt verwerkt.

Binnen de uitgewerkte oplossingen kwam Mastra als meest geschikte basis naar voren voor verdere uitwerking in de context van Springbok. De combinatie van inhoudelijke kwaliteit, betrouwbaarheid en voldoende aanvaardbare verwerkingstijd maakt deze aanpak het meest interessant voor praktische toepassing. Andere varianten bleken bruikbaar, maar boden minder consistente of minder volledige resultaten.

Deze conclusies moeten wel worden gelezen binnen de grenzen van de uitgevoerde evaluatie. De benchmark was beperkt in omvang en gebaseerd op een beperkt aantal representatieve testgevallen, waardoor verdere validatie nuttig blijft. Toekomstig onderzoek kan zich richten op een bredere evaluatieset, meerdere beoordelingsmodellen en een verdere integratie in een productieomgeving.

De bredere bijdrage van dit werk ligt in de demonstratie dat een agentgebaseerde aanpak meerwaarde kan bieden voor software-onderhoud en technische schuld. In plaats van beslissingen volledig over te nemen, ondersteunt het systeem ont-

wikkelaars door de interpretatielast te verkleinen en relevante informatie beter toegankelijk te maken.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Softwareprojecten binnen bedrijven en organisaties maken gebruik van talrijke externe bibliotheken, frameworks en tools, de zogenaamde *dependencies*. Deze dependencies worden voortdurend bijgewerkt om nieuwe functionaliteiten, verbeterde prestaties en vooral beveiligingspatches aan te bieden. (Pashchenko et al., 2018) In de praktijk blijkt het voor ontwikkelteams echter moeilijk om alle updates systematisch op te volgen en grondig te evalueren, zeker wanneer een bedrijf meerdere projecten en repositories beheert.

A.1.1. Probleemstelling

Het manueel opvolgen van dependency-updates vormt een groot probleem in softwareontwikkeling. Ontwikkelaars moeten regelmatig changelogs doorzoeken, compatibiliteit controleren en inschatten welke updates veilig kunnen worden doorgevoerd, wat in de praktijk veel tijd vraagt en vaak wordt uitgesteld. (Behutiye et al., 2024; Pashchenko et al., 2018) Onderzoek toont aan dat veel softwareprojecten hun dependencies niet systematisch bijhouden, waardoor een aanzienlijk deel verouderd raakt. Pashchenko et al. (Pashchenko et al., 2018) stellen bijvoorbeeld dat een belangrijk percentage kwetsbare dependencies relatief eenvoudig opgelost kan worden door een update, maar dat dit in de praktijk niet gebeurt wegens tijdsdruk en gebrek aan overzicht. Wanneer het updateproces niet systematisch gebeurt, stapelen onderhoudstaken zich op, waardoor het steeds moeilijker en tijdrovender wordt om de software up-to-date te houden. Dit fenomeen staat bekend als

technische schuld (technical debt): de achterstand die ontstaat doordat noodzakelijke verbeteringen of updates te lang worden uitgesteld, wat leidt tot hogere onderhoudskosten en complexiteit op lange termijn.(Behutiye et al., 2024; Ruiz et al., 2024) Empirisch onderzoek bevestigt dat technische schuld binnen bedrijven reële negatieve gevolgen heeft voor productiviteit en softwarekwaliteit.(Besker, 2020) Om deze last te verlichten, zetten veel organisaties al geautomatiseerde dependencybots in, zoals Dependabot¹ of Renovate.² Deze tools maken wel automatisch pull requests aan zodra er nieuwe versies beschikbaar zijn,(Erlenhov et al., 2022) maar geven doorgaans weinig context over de inhoud en impact van de wijziging. Ontwikkelaars verliezen daardoor nog steeds tijd aan het interpreteren van changelogs, het inschatten van risico's en het beslissen of een update al dan niet moet worden doorgevoerd.(Erlenhov et al., 2022) Deze bachelorproef richt zich daarom op het ontwikkelen van een *aparte webapplicatie* (een dashboard) die integreert met GitHub-repositories van het bedrijf. Per project houdt deze applicatie de gebruikte dependencies bij, controleert via publieke registries (zoals npm,³) of er nieuwe versies beschikbaar zijn en verzamelt de relevante changelogs en release notes. Een AI-agent, die in dit onderzoek *from scratch* wordt ontworpen op basis van bestaande AI-modellen maar zonder voorafgebouwd agentframework, analyseert deze informatie, genereert begrijpelijke samenvattingen en beoordeelt in welke mate een update cruciaal is (bijvoorbeeld om veiligheidsredenen) of eerder optioneel. De AI-agent geeft het kernteam via het dashboard inzicht in vragen zoals: welke dependencies zijn verouderd, wat is er precies veranderd in de nieuwe versie, zijn er bekende beveiligingsrisico's als niet wordt geüpdatet, en lijkt de update technisch laag- of hoogrisicovol. De webapplicatie leeft dus als een losstaand platform naast GitHub, maar gebruikt GitHub uitsluitend als bron voor project- en dependency-informatie en, eventueel, als kanaal om later pull requests of issues te creëren. Op die manier kan het updateproces slimmer, sneller en inzichtelijker worden gemaakt.

A.1.2. Onderzoeksvraag

Om het probleem van handmatig dependencybeheer en beperkte context bij updates op te lossen, wordt de volgende hoofdonderzoeksvraag geformuleerd:

Hoe kan een AI-gedreven agent worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren, zodat het kernteam dat verantwoordelijk is voor platform-

¹Dependabot is de ingebouwde GitHub-tool die kwetsbare of verouderde dependencies detecteert en automatisch update-pull-requests aanmaakt.<https://github.com/dependabot>

²Renovate is een open-source tool die automatisch update-pull-requests voor dependencies aanmaakt.<https://docs.renovatebot.com>

³npm is het standaard pakketbeheerplatform voor het JavaScript-ecosysteem.<https://www.npmjs.com>

en dependencybeheer efficiënter en met minder handmatig werk dependency-updates kan verwerken?

Hieruit vloeien de volgende deelvragen voort.

Deelvragen voor het beter begrijpen van het probleem

1. Wat zijn de belangrijkste uitdagingen en risico's bij het huidige, overwegend manuele dependency management voor het kernteam dat verantwoordelijk is voor platform- en dependencybeheer?
2. Welke bestaande tools en oplossingen voor automatisch dependencybeheer worden vandaag gebruikt, en welke sterke punten en beperkingen hebben deze oplossingen in de context van dit bedrijf?
3. Hoe leidt ophopende technische schuld op het vlak van dependency-updates ertoe dat er steeds meer tijd en inspanning nodig zijn voor het controleren en bijwerken van dependencies binnen het bedrijf?

Deelvragen richting de oplossing

4. Hoe kunnen AI-agents en workflow automatiseringsplatformen worden ingezet om informatie uit changelogs, release notes en dependency-pull-requests automatisch te interpreteren en in begrijpelijke vorm te communiceren naar het verantwoordelijke kernteam?
5. Welke functionele en niet-functionele vereisten moet een platform voor AI-gestuurd dependencybeheer vervullen, bijvoorbeeld op het vlak van integratie met versiebeheersystemen en registries, uitbreidbaarheid, beheer van regels en policies, auditlogging en performantie?
6. In welke mate voldoen geselecteerde AI-agent- of workflowplatformen aan deze vereisten, en hoe goed kunnen zij geïntegreerd worden in een proof-of-concept voor dependency management in de context van het bedrijf?

A.1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is om een proof-of-concept te ontwikkelen van een *aparte webapplicatie* (dashboard) die automatisch dependency-updates detecteert, analyseert en communiceert via een geïntegreerde, from-scratch ontworpen AI-agent, en om na te gaan in welke mate geselecteerde AI-agent- of workflowplatformen deze oplossing kunnen ondersteunen en voldoen aan de vereisten van het bedrijf voor geautomatiseerd dependencybeheer. Eerder onderzoek toont aan dat AI-technieken succesvol kunnen worden ingezet binnen onderhoudstaken zoals changelog-analyse en code review, wat aantoont dat deze toepassing haalbaar en relevant is. (Behutiye et al., 2024; Joshi, 2025; Kim et al., 2024; Zhu et al., 2025)

Concreet zal de oplossing:

- per project de gebruikte dependencies bijhouden;
- automatisch nagaan of er nieuwe versies beschikbaar zijn via publieke registers (zoals npm);
- changelogs en release notes analyseren met behulp van een AI-agent die samenvat wat er gewijzigd is en welke risico's of voordelen een update inhoudt;
- de verantwoordelijke developers via een dashboard informeren over de aard en impact van de updates, inclusief een korte, begrijpelijke samenvatting en een indicatie of de update cruciaal, risicovol of veilig lijkt;
- regels ondersteunen waardoor de developers gemakkelijker kan beslissen of en wanneer een dependency geüpdatet wordt, zonder dat deze beslissingen automatisch in GitHub worden afgedwongen.

De concrete technologiekeuzes worden in de verdere uitwerking van de bachelorproef gemotiveerd op basis van de requirements en de context van het bedrijf. Het onderzoek resulteert in:

- een prototype (demo) dat de werking van het systeem aantoont binnen een of enkele projecten van het bedrijf;
- een evaluatie van de toegevoegde waarde van AI bij dependency management voor de specifieke developer;
- een aanbeveling welk type platform, en eventueel welke concrete technologie, het meest geschikt is voor verdere toepassing binnen het bedrijf.

A.2. Literatuurstudie

A.2.1. Dependency management en kwetsbare dependencies

Moderne softwareprojecten steunen sterk op open-source dependencies, waardoor kwetsbaarheden in externe bibliotheken rechtstreeks een impact kunnen hebben op de veiligheid en betrouwbaarheid van toepassingen. (Pashchenko et al., 2018) Pashchenko et al. tonen aan dat een groot deel van de aangetroffen kwetsbare dependencies in de praktijk relatief eenvoudig verholpen kan worden door te upgraden naar een veiligere versie, maar dat ontwikkelteams deze updates vaak niet consequent doorvoeren. (Pashchenko et al., 2018)

In hun proefondervindelijk onderzoek onderscheiden Pashchenko et al. tussen daadwerkelijk gedeployde kwetsbare dependencies en libraries die enkel tijdens ontwikkeling of testing gebruikt worden, en stellen zij vast dat het merendeel van de reële kwetsbare dependencies door de eigen ontwikkelaars of directe onderhouders kan worden opgelost. (Pashchenko et al., 2018) Dit onderstreept het belang van systematisch dependencybeheer en duidelijke inzichten in welke dependencies effectief risico vormen in productie omgevingen. (Pashchenko et al., 2018)

A.2.2. Technische schuld en onderhoudslast

Het uitstellen van dependency-updates draagt bij aan de opbouw van technische schuld, waardoor onderhoud en toekomstige wijzigingen steeds meer inspanning vereisen.(Behutiye et al., 2024; Ruiz et al., 2024) Behutiye et al. analyseren de rol van technische schuld in agile omgevingen en beschrijven hoe keuzes die op korte termijn tijds winst opleveren, op langere termijn leiden tot hogere kosten en complexere wijzigingen.(Behutiye et al., 2024)

Ruiz et al. bespreken in hun tertiaire studie dat technische-schuldmanagement de voorbije jaren een groeiend onderzoeksdomein is geworden, waarbij organisaties worstelen met het in kaart brengen, prioriteren en terugdringen van opgebouwde schuld.(Ruiz et al., 2024) Empirisch werk van Chalmers University of Technology bevestigt dat technische schuld niet louter een theoretisch concept is, maar in industriële context concreet voelbaar is in de vorm van verminderde productiviteit, hogere foutkans en vertragingen in softwarelevering.(Besker, 2020)

A.2.3. Dependencybots en hun beperkingen

Om de handmatige last rond dependencybeheer te verlagen, zetten veel projecten dependency management bots in, zoals Dependabot en Renovate.(Erlenhov et al., 2022) Erlenhov et al. voeren een kwantitatieve en kwalitatieve studie uit naar dependencybots in open-source systemen en stellen vast dat slechts een beperkt aantal van de geanalyseerde geautomatiseerde tools voldoet aan de criteria van volwaardige “Devbots”, die autonoom bijdragen aan de codebasis.(Erlenhov et al., 2022)

Hun resultaten tonen dat projecten vaak experimenteren met meerdere bots en regelmatig wisselen, onder meer door problemen met ruis (te veel pull requests), beperkte configureerbaarheid en moeilijkheden om de impact van voorgestelde updates goed te begrijpen.(Erlenhov et al., 2022) Hoewel dependencybots het detecteren en aanmaken van update-pull-requests automatiseren, blijft de inhoudelijke beoordeling van changelogs, compatibiliteit en risico's grotendeels bij ontwikkelaars liggen, wat aansluit bij de probleemstelling van deze bachelorproef.(Erlenhov et al., 2022; Pashchenko et al., 2018)

A.2.4. AI, agents en workflow automatisering

Recente literatuur verkent hoe AI-agents en multi-agentframeworks kunnen worden ingezet om complexe taken in softwareontwikkeling en data-intensieve workflows te coördineren.(Joshi, 2025; Kim et al., 2024; Zhu et al., 2025) Joshi biedt een overzicht van autonome systemen en collaboratieve AI-agentframeworks en benadrukt dat zulke agents vooral nuttig zijn in scenario's met veel herhalende beslissingen op basis van tekstuele en gestructureerde input, zoals logs, notificaties en rapporten.(Joshi, 2025)

Kim et al. introduceren met Bel Esprit een multi-agentframework voor het opbou-

wen van AI-model-pijplijnen, waarbij verschillende gespecialiseerde agents samenwerken om complexere taken modulair af te handelen.(Kim et al., 2024) Zhu et al. bestuderen in OAgents empirisch welke ontwerpkeuzes leiden tot effectieve agents, en bespreken onder meer het belang van duidelijke taakafbakening, robuuste toolintegratie en feedbackloops voor betrouwbare beslissingsondersteuning.(Zhu et al., 2025) Deze inzichten zijn relevant voor het ontwerp van een AI-agent die dependency-updates interpreteert, samenvat en beslissingsvoorstellen formuleert voor een onderhoudsteam.

Naast generieke agentframeworks beschrijft Wali et al. hoe workflow-automatisering met n8n⁴ operationele efficiëntie kan verhogen door repetitieve, gegevensgedreven processen te coördineren over verschillende systemen heen.(Wali et al., 2025) Hoewel hun casus zich richt op een financiële context, illustreert de studie dat low-code workflowtools geschikt zijn om integraties met externe APIs, databronnen en notificatiekanalen te realiseren, wat ook essentieel is voor een geautomatiseerde dependency-updatepipeline.(Wali et al., 2025)

A.2.5. Onderzoeksniche: AI-ondersteund dependencybeheer

Samengenomen schetst de literatuur een duidelijk beeld: kwetsbare of verouderde dependencies vormen een reëel risico, maar kunnen vaak verholpen worden door updates die vandaag onvoldoende systematisch worden uitgevoerd.(Behutiye et al., 2024; Pashchenko et al., 2018) Tegelijk tonen studies naar technische schuld dat het structureel uitstellen van onderhoud, waaronder dependency updates, leidt tot een groeiende onderhoudslast en achterstand.(Behutiye et al., 2024; Besker, 2020; Ruiz et al., 2024)

Onderzoek naar dependencybots maakt duidelijk dat huidige oplossingen weliswaar updates detecteren en pull requests genereren, maar ontwikkelaars nog steeds belasten met het interpreteren van changelogs en het beoordelen van risico's en prioriteiten.(Erlenhov et al., 2022) De recente vooruitgang in AI-agents en workflow-automatisering suggereert dat een volgende stap mogelijk is: een systeem waarin een AI-agent niet alleen updates signaleert, maar ook de inhoud van release notes en changelogs analyseert, samenvat en contextafhankelijk advies geeft aan een klein kernteam dat verantwoordelijk is voor dependencybeheer.(Joshi, 2025; Kim et al., 2024; Wali et al., 2025; Zhu et al., 2025)

Deze bachelorproef positioneert zich precies in deze niche door een AI-gestuurde laag bovenop bestaande dependencybots te onderzoeken en te prototypen, met de expliciete focus op het verlagen van de manuele beoordelingslast en het beheersbaar houden van technische schuld rond dependencies in een bedrijfscontext.

⁴n8n is een open-source tool om workflows en API's visueel te automatiseren.<https://n8n.io>

A.3. Methodologie

Dit onderzoek volgt de opzet van een vergelijkende studie met proof-of-concept, aangevuld met literatuuronderzoek en een requirements-analyse binnen een concrete bedrijfscontext. Het doel is om enerzijds het probleem en de oplossingsruimte rond dependencybeheer in kaart te brengen, en anderzijds een apart dashboard-prototype te bouwen en technisch te evalueren binnen een realistische ontwikkelomgeving.

A.3.1. Fase 1 - Verdiepende literatuurstudie en probleemverkenning

In de eerste fase van de bachelorproef wordt de probleemcontext verder uitgediept aan de hand van een systematische literatuurstudie rond dependency management, kwetsbare dependencies, technische schuld en bestaande dependencybots. Daarnaast wordt vakliteratuur over AI-agents en workflow-automatisering bestudeerd om na te gaan welke concepten en benaderingen relevant zijn voor een AI-gestuurd dependency-dashboard.

Tijdens deze fase worden ook één of enkele representatieve projecten van het bedrijf geselecteerd en geanalyseerd (projectstructuur, gebruikte technologieën, dependency-ecosystemen, huidige processen rond updates). De belangrijkste deliverables zijn:

- een uitgewerkte en geactualiseerde literatuurstudie;
- een scherpere probleemomschrijving toegespitst op de gekozen bedrijfscontext en projecten;
- een eerste set geïnformeerde hypothesen over de verwachte impact van een AI-gestuurd dependency-dashboard.

A.3.2. Fase 2 - Requirements-analyse en architectuurontwerp

In de tweede fase wordt eerst een gerichte requirements-analyse uitgevoerd in samenwerking met de co-promotor via interviews en workshops worden functionele en niet-functionele eisen expliciet gemaakt, zoals:

- functionele eisen: integratie met GitHub, beheer van projecten en dependencies in het dashboard, detectie van nieuwe versies via publieke registries, analyse en samenvatting van changelogs en release notes, aanduiding van kriticiiteit (security, bugfix, feature), prioritering van updates;
- niet-functionele eisen: schaalbaarheid, betrouwbaarheid, traceerbaarheid (logging en audittrail), gebruiksvriendelijkheid en onderhoudbaarheid.

Op basis van deze requirements wordt vervolgens een systeemarchitectuur ontworpen voor de webapplicatie en de from-scratch ontworpen AI-agent. De architectuur beschrijft onder meer de globale componenten (frontend, backend, databank, integraties met GitHub en registries), de verantwoordelijkheden per component, de datastromen (bijvoorbeeld van dependency-informatie naar AI-analyse en

terug naar het dashboard) en de plaats van eventueel ondersteunende workflow- of agentplatformen. Deliverables van fase 2 zijn:

- een requirementsdocument met duidelijke prioritering;
- een architectuurbeschrijving (diagrammen, componentbeschrijvingen, data-modellen);
- een lijst van geselecteerde technologieën en tools, gemotiveerd vanuit de requirements.

A.3.3. Fase 3 - Implementatie van het proof-of-concept

In fase 3 wordt de ontworpen architectuur omgezet in een werkend proof-of-concept. De kern bestaat uit een aparte webapplicatie (dashboard) waarin per project de gebruikte dependencies worden bijgehouden en verrijkt met informatie uit publieke registries zoals npm. De applicatie werkt vanaf het begin met één of meerdere reële GitHub-repositories van het bedrijf, zodat de proof-of-concept onmiddellijk getest wordt in een realistische context. Dependency-informatie wordt automatisch uit deze repositories gehaald (bijvoorbeeld via repositoryconfiguratiebestanden) en opgeslagen in een centrale databank.

Parallel wordt de AI-agentlaag geïmplementeerd in de backend. In deze fase worden drie concrete AI-agentoplossingen opgezet en geïntegreerd in hetzelfde dashboard: een agent op basis van OpenAI Agent Builder, een workflow met n8n en een agentconfiguratie met Flowise. Elk van deze varianten ontvangt changelogs en release-note-informatie, genereert beknopte samenvattingen en beoordeelt de aard en vermoedelijke impact van updates (bijvoorbeeld beveiligingsfix, bugfix, nieuwe functionaliteit, potentieel brekende wijziging). De frontend visualiseert de status van dependencies, aanbevolen acties en de door de verschillende AI-varianten gegenereerde toelichting. Deliverables van fase 3 zijn:

- een werkend prototype van het dashboard met integratie naar GitHub en één dependency-registries;
- drie geïmplementeerde AI-agentvarianten (OpenAI Agent Builder, n8n, Flowise) die changelogs en release notes verwerken en samenvatten;
- technische documentatie over de implementatie (architectuur, API's, datamodellen en integratiepunten voor de agents).

A.3.4. Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak

De vierde fase richt zich op het onderbouwen en verfijnen van de gemaakte ontwerpkeuzes aan de hand van een gerichte vergelijking van de drie uitgewerkte AI-agentoplossingen en eventuele architecturale varianten. Op basis van de requi-

rements uit fase 2 wordt een set *eenvoudig meetbare* evaluatiecriteria opgesteld, zoals:

- gemiddelde verwerkingstijd per update (doorlooptijd van input tot samenvatting);
- technische inspanning voor opzet en onderhoud (bijvoorbeeld aantal configuratiestappen, hoeveelheid code);
- kwaliteit van samenvattingen gemeten via een eenvoudige beoordelingsschaal door de co-promotor (bijvoorbeeld 1–5 voor duidelijkheid en relevantie);
- fout- of faalratio (bijvoorbeeld aantal mislukte runs of onvolledige resultaten per tien updates).

Aan de hand van een aantal representatieve scenario's (bijvoorbeeld updates met alleen kleine bugfixes, updates met beveiligingspatches, updates met potentieel brekende wijzigingen) worden OpenAI Agent Builder, n8n en Flowise onder dezelfde omstandigheden getest en met elkaar vergeleken. De focus ligt hierbij niet op een brede marktanalyse, maar op het versterken of bijsturen van de gekozen architectuur en het bepalen welke aanpak het meest geschikt is als basis voor het verdere gebruik van het dashboard in deze specifieke use case. Deliverables van fase 4 zijn:

- experimentele beschrijvingen van de uitgeteste varianten;
- meetresultaten volgens de gedefinieerde criteria (doorlooptijd, inspanning, beoordelingsscores, foutpercentages);
- een gemotiveerde keuze of bevestiging van de uiteindelijke architectuur en AI-configuratie.

A.3.5. Fase 5 - Evaluatie en rapportering

In de laatste fase wordt het proof-of-concept geëvalueerd met de co-promotor binnen het bedrijf, aan de hand van een combinatie van demonstraties, gericht feedbackgesprekken en, waar mogelijk, beperkte praktijktesten realistische projecten. De evaluatie focust op bruikbaarheid van het dashboard, de gemeten vermindering van manuele analysetijd (bijvoorbeeld aantal geanalyseerde updates per uur vóór en na gebruik), de door de co-promotor toegekende beoordelingsscores voor duidelijkheid van de AI-samenvattingen en de mate waarin het systeem helpt bij de prioritering van updates.

De bevindingen uit deze evaluatie worden gecombineerd met de kwantitatieve resultaten van fase 4 om de onderzoeksvragen te beantwoorden en de onderzoekshypothesen te toetsen. Deliverables van fase 5 zijn:

- een synthese van de evaluatie met de co-promotor (inclusief gemeten tijdswinst, kwaliteitsbeoordelingen en verbetervoorstellen);
- de uitgewerkte bachelorproef met een geïntegreerde beschrijving van probleem, methode, resultaten en conclusies;
- aanbevelingen voor verdere ontwikkeling en mogelijke integratie van het systeem in de bredere ontwikkelprocessen van het bedrijf.

De globale fasering en tijdsplanning van het onderzoek wordt weergegeven in Figuur A.1.

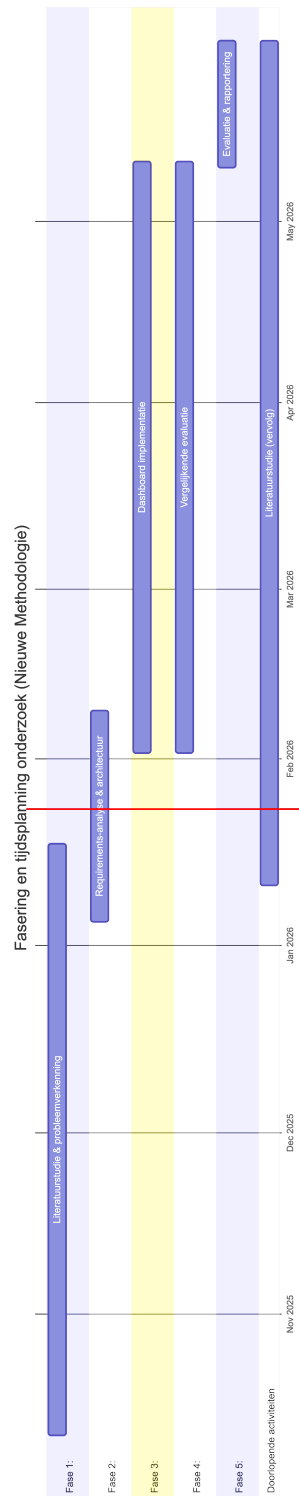
A.4. Verwacht resultaat en conclusie

Op basis van de probleemstelling en onderzoeksdoelstelling wordt verwacht dat het proof-of-concept in staat zal zijn om dependency-updates automatisch te detecteren, te analyseren en in een compacte, begrijpelijke vorm te presenteren. De AI-agent zal changelogs en release notes samenvatten, het type wijziging (bijvoorbeeld beveiligingsfix, bugfix of nieuwe functionaliteit) aanduiden en een indicatie geven van de vermoedelijke impact op het project.

Verder wordt verwacht dat de werklast daalt doordat een deel van de beslissingen rond relatief veilige, backward compatible updates (zoals patch- of beperkte minor-releases) eenvoudiger of gedeeltelijk geautomatiseerd kan worden, terwijl potentieel risicovolle major-updates expliciet voor manuele review gemarkeerd blijven. Indien er voldoende meetdata beschikbaar komt (bijvoorbeeld logbestanden of tijdsregistraties), kan een eerste, kwantitatieve inschatting gemaakt worden van de tijdswinst in vergelijking met de huidige, hoofdzakelijk manuele aanpak.

De doelgroep, haalt hieruit meerwaarde doordat zij sneller prioriteiten kunnen bepalen, minder tijd verliezen aan het doorpluizen van changelogs en release notes en een beter overzicht krijgen van de actuele staat van dependencies en de bijhorende technische schuld. Daarnaast ontstaat een herhaalbaar proces dat later kan opgeschaald worden naar andere teams of projecten binnen de organisatie.

Verwacht wordt dat de evaluatie van de gekozen AI-agent- en/of workflowaanpak zal tonen dat geen enkele oplossing op alle criteria de beste is, maar dat duidelijke verschillen zichtbaar worden op het vlak van integratiemogelijkheden, gebruiksgemak, onderhoudbaarheid en performantie. Op basis van deze inzichten kan een onderbouwde aanbeveling worden geformuleerd voor een architectuur en toolkeuze die het best aansluit bij de context van het bedrijf, en kan geconcludeerd worden dat een AI-gestuurde laag een zinvolle stap is in het beperken van technische schuld rond dependencies. Indien de uiteindelijke resultaten afwijken van deze verwachtingen, biedt dit aanleiding om te onderzoeken welke aannames niet bleken te kloppen en welke randvoorwaarden essentieel zijn voor succesvol AI-ondersteund dependencybeheer.



Figuur A.1: Fasering en tijdsplanning van het onderzoek

Bibliografie

- Anthropic. (2025). Tool Use (Function Calling). Geraadpleegd op 4 mei 2026, van <https://docs.anthropic.com/en/docs/build-with-claude/tool-use>
- Behutiye, W. N., Rodriguez, P., Oivo, M., & Tosun, A. (2024). Analyzing the Concept of Technical Debt in the Context of Agile Software Development: A Systematic Literature Review. *arXiv*. <https://arxiv.org/abs/2401.14882>
- Besker, T. (2020). *Technical Debt: An Empirical Investigation of Its Harmfulness and Management Strategies in Industry* [PhD thesis]. Chalmers University of Technology. <https://research.chalmers.se/en/publication/518318>
- Devon Sun. (2025). *Mastra: Build Powerful AI Agents with Typescript* [Tutorial met voorbeeldagents, tools en workflows in Mastra]. The AI Builders. Geraadpleegd op 2 maart 2026, van https://tutorial.theaibuilders.dev/tutorials/Frameworks/mastra_tutorial
- Erlenhov, L., De Oliveira Neto, F. G., & Leitner, P. (2022). Dependency Management Bots in Open-Source Systems—Prevalence and Adoption. *PeerJ Computer Science*, 8, e849. <https://doi.org/10.7717/peerj-cs.849>
- Hatchworks. (2025). *n8n Guide 2026: Features & Workflow Automation Deep Dive* [Diepgaande gids over n8n met nadruk op AI-workflows, triggers en integraties]. Geraadpleegd op 9 maart 2026, van <https://hatchworks.com/blog/ai-agents/n8n-guide/>
- He, R., He, H., Zhang, Y., & Zhou, M. (2023). Automating Dependency Updates in Practice: An Exploratory Study on GitHub Dependabot. *IEEE Transactions on Software Engineering*, 49(8), 4004–4022. <https://doi.org/10.1109/TSE.2023.3278129>
- Joshi, S. (2025). Review of Autonomous Systems and Collaborative AI Agent Frameworks. *International Journal of Science and Research Archive*, 14(02), 961–972. <https://doi.org/10.30574/ijrsra.2025.14.2.0439>
- Kim, Y., Abdelaziz, A. E., Castro Ferreira, T., Al-Badrashiny, M., & Sawaf, H. (2024). Bel Esprit: Multi-Agent Framework for Building AI Model Pipelines. *arXiv*. <https://doi.org/10.48550/arXiv.2412.14684>
- Langflow. (2025). The Complete Guide to Choosing an AI Agent Framework in 2025. Geraadpleegd op 4 mei 2026, van <https://www.langflow.org/blog/the-complete-guide-to-choosing-an-ai-agent-framework-in-2025>

- MastraAI. (2025). *Observability Overview* [Documentatie over tracing en observability voor Mastra-agents en workflows]. Geraadpleegd op 2 maart 2026, van <https://mastra.ai/docs/observability/overview>
- MastraAI. (2026). *Mastra: The TypeScript AI Agent Framework* [Open-source framework voor het bouwen van AI-agents en workflows in TypeScript]. Geraadpleegd op 2 maart 2026, van <https://github.com/mastra-ai/mastra>
- n8n. (2025). *n8n Documentation: About n8n* [Officiële documentatie over n8n als workflow-automatiseringsplatform]. Geraadpleegd op 9 maart 2026, van <https://docs.n8n.io>
- Nair, P. (2025). The Ultimate Guide to Agentic AI Frameworks in 2025: Code vs. No-Code Showdown. *Medium*. Geraadpleegd op 4 mei 2026, van <https://medium.com/@pranavnairop090/the-ultimate-guide-to-agentic-ai-frameworks-in-2025-code-vs-no-code-showdown-542a8570eb8e>
- OpenAI. (2025). OpenAI Agents SDK. Geraadpleegd op 4 mei 2026, van <https://platform.openai.com/docs/guides/agents>
- OpenAI. (2026a). Agents SDK. Geraadpleegd op 20 april 2026, van <https://developers.openai.com/api/docs/guides/agents>
- OpenAI. (2026b). Function calling | OpenAI API. Geraadpleegd op 20 april 2026, van <https://developers.openai.com/api/docs/guides/function-calling>
- OpenAI. (2026c). Using tools | OpenAI API. Geraadpleegd op 20 april 2026, van <https://developers.openai.com/api/docs/guides/tools>
- Pashchenko, I., Plate, H., Ponta, S. E., Sabetta, A., & Massacci, F. (2018). Vulnerable Open Source Dependencies: Counting Those That Matter. *Proceedings of the 12th International Symposium on Empirical Software Engineering and Measurement (ESEM)*.
- Proser, Z. (2025). *n8n: The Workflow Automation Tool for the AI Age* [Artikel dat de architectuur, event-gedreven werking en AI-toepassingen van n8n beschrijft]. Geraadpleegd op 9 maart 2026, van <https://workos.com/blog/n8n-the-workflow-automation-tool-for-the-ai-age>
- Ruiz, J. A., Aguilar, R. A., Garcilazo, J. F., & Aguilera, A. A. (2024). A Tertiary Study on Technical Debt Management over the last lustrum. *International Journal of Combinatorial Optimization Problems and Informatics*, 15(5), 181–192. <https://doi.org/10.61467/2007.1558.2024.v15i5.577>
- Tavily. (2026a). *What is the Tavily Search API?* [Help center article]. Geraadpleegd op 27 april 2026, van <https://help.tavily.com/articles/4840311948-tavily-search-api>
- Tavily. (2026b, februari). *Best Practices for Search* [Tavily documentatie]. Geraadpleegd op 27 april 2026, van <https://docs.tavily.com/documentation/best-practices/best-practices-search>

- Tavily. (2026c, februari). *Introduction* [Tavily API reference]. Geraadpleegd op 27 april 2026, van <https://docs.tavily.com/documentation/api-reference/introduction>
- Tavily. (2026d, april). *Web Search Essentials* [Tavily documentation]. Geraadpleegd op 27 april 2026, van <https://docs.tavily.com/examples/quick-tutorials/search-api>
- Tavily AI. (2025, december). *Introduction* [Alternative Tavily documentation mirror]. Geraadpleegd op 27 april 2026, van <https://tavilyai.mintlify.app/documentation/api-reference/introduction>
- Turing. (2026). A Detailed Comparison of Top 6 AI Agent Frameworks in 2026. Geraadpleegd op 4 mei 2026, van <https://www.turing.com/resources/ai-agent-frameworks>
- Wali, M., Nasir, N., & Iqbal, T. (2025). Implementing Workflow Automation with n8n to Enhance Operational Efficiency and Performance in the Sharia Cooperative of Bank Indonesia, Aceh Province. *Journal Digital Technology Trend*, 4(1), 36–47. <https://doi.org/10.56347/jdtt.v4i1.341>
- Zhu, H., Qin, T., Zhu, K., Huang, H., Guan, Y., Xia, J., Yao, Y., Li, H., Wang, N., Liu, P., Peng, T., Gui, X., Li, X., Liu, Y., Jiang, Y. E., Wang, J., Zhang, C., Tang, X., Zhang, G., ... Zhou, W. (2025). OAgents: An Empirical Study of Building Effective Agents. <https://arxiv.org/abs/2506.15741>